

■ ZERO TO HERO · 200 PAGES

Complete Python Course

From Beginner to Advanced — 200 Pages of Real Python

COURSE CONTENTS

■ Basics, Variables, Operators

■■ Functions & Scope

■ File I/O & Exceptions

■ Modules & Packages

■ DSA & Algorithms

■ Control Flow & Loops

■ Strings & Collections

■■ Object-Oriented Python

■ Advanced Python

■ Projects & Practice

About This Course

This 200-page course takes you from absolute zero to advanced Python programming. Whether you have never written a line of code or you know the basics and want to go deeper — this course is structured to take you all the way.

Who Is This For?

- Complete beginners with no prior programming experience
- Students who know another language and want to learn Python fast
- Freshers preparing for MNC interviews (TCS, Infosys, Wipro, Amazon)
- Anyone who wants to use Python for data science, web, or automation

How to Use This Course

- Read every page in order — concepts build on each other
- Type every code example by hand — do not copy-paste
- Complete every exercise before moving to the next chapter
- Revisit any chapter when you get stuck — it is your reference too

What You Will Build

- Calculator (Chapter 7)
- Number Guessing Game (Chapter 6)
- To-Do List Manager with File Storage (Chapter 11)
- Student Grade Manager with OOP (Chapter 13)

■ COURSE TIP:

The best way to learn Python is to write code every single day — even 20 minutes. By page 200, you will have written more than 2,000 lines of Python.

Table of Contents

Chapter 1 Introduction to Python **pp. 8–19**

Chapter 2 Variables & Data Types **pp. 20–32**

Chapter 3 Operators & Expressions **pp. 33–43**

Chapter 4 Strings **pp. 44–56**

Chapter 5 Control Flow **pp. 57–69**

Chapter 6 Loops **pp. 70–81**

Chapter 7 Functions **pp. 82–96**

Chapter 8 Lists **pp. 97–108**

Table of Contents (cont.)

Chapter 9 **Tuples & Sets** **pp. 109–118**

Chapter 10 **Dictionaries** **pp. 119–130**

Chapter 11 **File Handling** **pp. 131–142**

Chapter 12 **Exception Handling** **pp. 143–152**

Chapter 13 **Object-Oriented Programming** **pp. 153–170**

Chapter 14 **Modules & Packages** **pp. 171–180**

Chapter 15 **Advanced Python** **pp. 181–194**

Chapter 16 **DSA & Projects** **pp. 195–200**

Introduction to Python

Your first steps into the world of Python programming

What you will learn:

■ What is Python?

■ Installing Python

■ Running your first program

■ Writing .py files

■ Why Python is popular

■ Python vs other languages

■ Python interactive mode

■ Comments & PEP8

Chapter 1 · Complete Python Course

What Is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It is designed for simplicity and readability — Python code often reads like plain English.

Why Python?

- Easy to learn — minimal syntax, maximum readability
- Interpreted — no compilation step required, runs directly
- Cross-platform — works on Windows, macOS, Linux
- Huge standard library — 'batteries included'
- Massive community — millions of developers worldwide
- Used everywhere: Web, AI/ML, Data Science, Automation, Scripting

Where Is Python Used?

- Google, YouTube, Instagram, Netflix, NASA all use Python
- Machine learning libraries: TensorFlow, PyTorch, scikit-learn
- Web frameworks: Django, Flask, FastAPI
- Data analysis: pandas, NumPy, Matplotlib
- Automation & scripting: file management, bots, scrapers

■ CAREER FACT:

Python is the #1 most popular language on GitHub, Stack Overflow, and TIOBE Index. Learning it opens doors to Data Science, AI, Backend Web, and Automation roles.

Installing Python & Your First Program

Installing Python

- Go to python.org/downloads — download Python 3.11 or higher
- During install on Windows: CHECK 'Add Python to PATH'
- Verify: open terminal/cmd and type: `python --version`

```
# Verify Python is installed
# In terminal / command prompt:
$ python --version
Python 3.11.4

# Start interactive Python shell:
$ python
>>> print('Hello, World!')
Hello, World!
>>> 2 + 3
5
```

Your First Python File

Create a file called `hello.py` and write your first program. Save it and run it from the terminal with: `python hello.py`

```
# hello.py – your first Python program

print('Hello, World!')      # print to screen
print('My name is Python')  # another line
print(2 + 3)                # arithmetic works too
print('Python version: 3.11') # string output

# Output:
# Hello, World!
# My name is Python
# 5
# Python version: 3.11
```

■ RECOMMENDED TOOLS:

VS Code (free) + Python extension is the best setup for beginners. Install from code.visualstudio.com. JupyterLab is great for data science.

Comments, Indentation & PEP8

Python uses indentation (spaces/tabs) to define blocks of code — unlike Java or C which use braces {}. This forces clean, readable code. Use 4 spaces per indent level. PEP8 is Python's official style guide.

Comments

```
# This is a single-line comment

'''
This is a
multi-line comment (docstring)
'''

x = 5    # inline comment after code

# Good practice: comment WHY, not WHAT
# BAD:   x = x + 1    # adds 1 to x
# GOOD: x = x + 1    # move to next page
```

Indentation Rules

```
# 4 spaces = 1 indent level
if True:
    print('indented – inside if block')
    print('still inside if')
print('outside if block – no indent')

# WRONG – mixed indent causes IndentationError:
# if True:
#     print('wrong – only 1 space')
```

PEP8 Quick Rules

- Use 4 spaces (not tabs) for indentation
- Max 79 characters per line
- 2 blank lines between functions/classes
- 1 blank line between methods inside a class
- Variable names: lowercase_with_underscores

The print() Function

print() is the most basic output function in Python. It displays output to the screen (standard output). Understanding its options helps write cleaner output.

```
# Basic print
print('Hello, World!')
print(42)
print(3.14)

# Multiple values
print('Name:', 'Rohan', 'Age:', 22)
# Output: Name: Rohan Age: 22

# sep parameter – custom separator
print('a', 'b', 'c', sep='-')
# Output: a-b-c

# end parameter – what to print at end
print('Hello', end=' ')
print('World')    # Output: Hello World

# print to file
import sys
print('Error!', file=sys.stderr)
```

print() with Numbers

```
x = 10; y = 3.14159
print(x, y)           # 10 3.14159
print(f'x={x}, y={y:.2f}') # x=10, y=3.14
print('Sum:', x + int(y)) # Sum: 13
```

■ QUICK TIP:

print() with f-strings (formatted string literals) is the modern, preferred way to format output. We cover f-strings in depth in Chapter 4.

Python Keywords & Identifiers

Reserved Keywords (cannot use as variable names)

```
# Python's 35 reserved keywords:
False    await    else    import    pass
None     break    except   in         raise
True     class   finally is         return
and      continue for      lambda    try
as       def     from     nonlocal  while
assert   del     global   not        with
async    elif    if       or         yield
```

Naming Rules for Identifiers

- Must start with a letter (a-z, A-Z) or underscore _
- Cannot start with a digit (1name is INVALID)
- Can contain letters, digits, and underscores
- Case-sensitive: age, Age, and AGE are 3 different variables
- Cannot be a reserved keyword

```
# VALID identifiers
name = 'Rohan'
my_age = 22
_private = 'hidden'
myVar2 = 100
MAX_SIZE = 500

# INVALID identifiers
# 2name = 'bad'           # starts with digit
# my-name = 'bad'         # hyphen not allowed
# class = 'bad'           # reserved keyword
# my name = 'bad'         # space not allowed
```

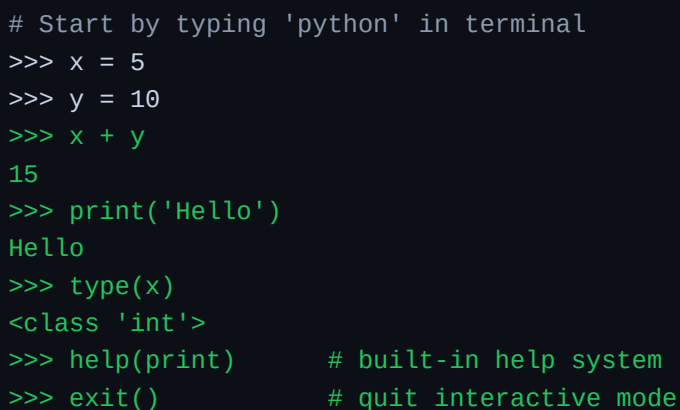
Naming Conventions

- Variables & functions: snake_case (my_variable)
- Constants: UPPER_SNAKE_CASE (MAX_SIZE)

Interactive Mode vs Script Mode

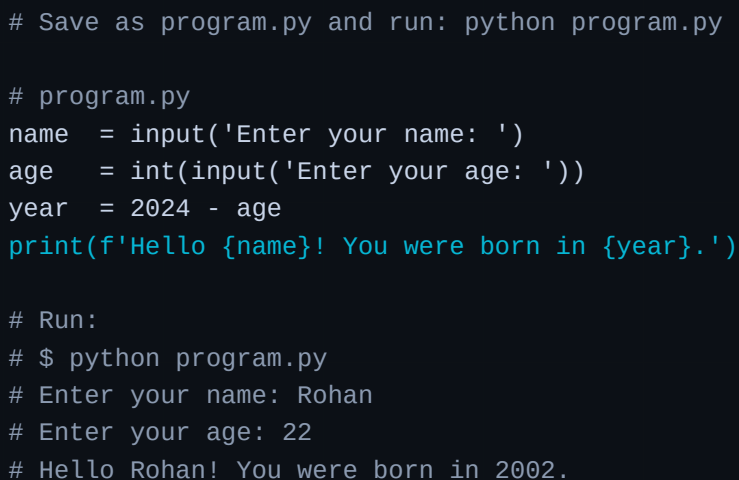
Python can be run in two modes: Interactive mode (the REPL) is great for experimenting. Script mode runs .py files and is how real programs are written.

Interactive Mode (REPL)



```
# Start by typing 'python' in terminal
>>> x = 5
>>> y = 10
>>> x + y
15
>>> print('Hello')
Hello
>>> type(x)
<class 'int'>
>>> help(print)      # built-in help system
>>> exit()           # quit interactive mode
```

Script Mode



```
# Save as program.py and run: python program.py

# program.py
name = input('Enter your name: ')
age = int(input('Enter your age: '))
year = 2024 - age
print(f'Hello {name}! You were born in {year}.')

# Run:
# $ python program.py
# Enter your name: Rohan
# Enter your age: 22
# Hello Rohan! You were born in 2002.
```

IDE / Jupyter Notebook

- VS Code: best for general Python development
- PyCharm: feature-rich, great for large projects
- Jupyter Notebook/Lab: best for data science & exploration

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Print your name, age, and favourite language

```
# Exercise 1
print('Name: Rohan')
print('Age:', 22)
print('Favourite language: Python')
```

Exercise 2: Use sep and end in print()

```
# Exercise 2
print('Python', 'is', 'awesome', sep=' >>> ')
print('Line 1 ', end='')
print('Line 2') # same line as above
```

Exercise 3: Write a program that asks for your name and greets you

```
# Exercise 3
name = input('What is your name? ')
print('Hello,', name, '! Welcome to Python.')
```

Exercise 4: Check if 'class' is a keyword using the keyword module

```
import keyword
print(keyword.iskeyword('class')) # True
print(keyword.iskeyword('myvar')) # False
print(keyword.kwlist)             # all keywords
```

Variables & Data Types

Understanding how Python stores and represents data

What you will learn:

- Variables & assignment

- str (strings)

- type() & isinstance()

- None type

- int, float, complex

- bool (True/False)

- Type conversion

- Dynamic typing

Chapter 2 · Complete Python Course

Variables & Assignment

A variable is a name that refers to a value stored in memory. Python is dynamically typed — you do not declare the type; Python figures it out at runtime.

```
# Simple assignment
name  = 'Rohan'      # str
age    = 22           # int
height = 5.9         # float
is_student = True     # bool

# Multiple assignment
x = y = z = 0         # all equal 0

# Tuple unpacking / destructuring
a, b, c = 1, 2, 3
print(a, b, c)        # 1 2 3

# Swap variables (Pythonic!)
a, b = b, a
print(a, b)           # 2 1

# Augmented assignment
score = 100
score += 10            # score = score + 10 → 110
score -= 5             # 105
score *= 2             # 210
score //= 3            # 70
```

Variable Memory

- Variables store a reference (pointer) to an object, not the value itself
- Two variables can point to the same object (aliasing)
- `id()` returns the memory address of an object
- `del` variable removes the binding (not necessarily the object)

■ PYTHON FACT:

Python caches small integers (-5 to 256). So `a=5; b=5; a is b` returns `True`. For large numbers, `a is b` may be `False` even if values are equal.

Numbers — int & float

Python has three numeric types: int (integers), float (decimal numbers), and complex (real + imaginary). Python 3 integers have unlimited precision — they can be arbitrarily large.

Integers (int)

```
# Integer literals
x = 42
y = -100
big = 1_000_000          # underscores for readability
binary = 0b1010         # binary → 10
octal = 0o17             # octal → 15
hexa = 0xFF              # hex → 255

# Integer operations
print(10 // 3)           # 3 (floor division)
print(10 % 3)            # 1 (modulo / remainder)
print(2 ** 10)           # 1024 (power)
print(abs(-42))          # 42 (absolute value)
```

Floats (float)

```
# Float literals
pi = 3.14159
small = 1.5e-3            # scientific notation → 0.0015
large = 2.5e6             # 2500000.0

# Float precision warning!
print(0.1 + 0.2)          # 0.30000000000000004 (!)
print(round(0.1+0.2, 2)) # 0.3 (use round())

# Float operations
import math
print(math.floor(3.9))    # 3
print(math.ceil(3.1))     # 4
print(math.sqrt(16))      # 4.0
print(math.pi)           # 3.141592653589793
```

■ ■ FLOAT WARNING:

Floats have precision issues due to binary representation. For financial calculations, use the decimal module: `from decimal import Decimal`

Booleans & None

Boolean (bool)

bool is a subclass of int in Python. True equals 1 and False equals 0. This leads to some useful but surprising behaviours.

```
# Boolean values
is_active = True
has_error = False

# bool is subclass of int
print(True + True)      # 2
print(True * 5)         # 5
print(False + 1)        # 1

# Truthy and Falsy values
# Falsy: 0, 0.0, '', [], {}, (), None, False
# Everything else is Truthy
print(bool(0))           # False
print(bool(''))          # False
print(bool([]))          # False
print(bool(42))          # True
print(bool('hello'))     # True
print(bool([1,2,3]))     # True
```

None Type

None represents the absence of a value. It is similar to null in other languages. Functions that do not explicitly return a value return None.

```
# None
x = None
print(x)                 # None
print(type(x))           # <class 'NoneType'>

# Check for None – always use 'is', not '=='
if x is None:
    print('x has no value')

# Functions return None implicitly
def no_return():
    pass
result = no_return()
print(result)            # None
```

Type Conversion (Casting)

Python allows converting between types using built-in functions. Implicit conversion (coercion) happens automatically in some cases. Explicit conversion (casting) is done manually.

Explicit Type Conversion

```
# int() – convert to integer
print(int('42'))          # 42
print(int(3.9))            # 3 (truncates, no rounding!)
print(int(True))           # 1
print(int('0xFF', 16))    # 255 (hex string to int)

# float() – convert to float
print(float('3.14'))       # 3.14
print(float(10))            # 10.0
print(float(True))          # 1.0

# str() – convert to string
print(str(42))              # '42'
print(str(3.14))            # '3.14'
print(str(True))            # 'True'
print(str(None))            # 'None'

# bool() – convert to boolean
print(bool(1))              # True
print(bool(0))              # False
print(bool('Python'))       # True
print(bool(''))              # False
```

Common Conversion Pitfalls

```
# int() fails on non-numeric strings
# int('hello') → ValueError!
# int('3.14') → ValueError! (use float first)
print(int(float('3.14'))) # 3 (correct way)

# Safe conversion with try/except
try:
    x = int(input('Enter a number: '))
except ValueError:
    print('That was not a number!')
```

type() and isinstance()

type() returns the exact type of an object. isinstance() checks if an object is an instance of a class (or a tuple of classes). isinstance() is preferred in production code.

```
# type() – exact type
print(type(42))           # <class 'int'>
print(type(3.14))         # <class 'float'>
print(type('hello'))      # <class 'str'>
print(type([1,2,3]))       # <class 'list'>
print(type(True))         # <class 'bool'>
print(type(None))         # <class 'NoneType'>

# Check type with ==
x = 42
if type(x) == int:
    print('x is an integer')    # prints

# isinstance() – preferred
print(isinstance(42, int))      # True
print(isinstance(42, float))    # False
print(isinstance(True, int))    # True (bool is int subclass!)
print(isinstance('hi', (int,str))) # True – check multiple types

# isinstance with class hierarchy
class Animal: pass
class Dog(Animal): pass
d = Dog()
print(isinstance(d, Dog))       # True
print(isinstance(d, Animal))    # True (subclass!)
print(type(d) == Animal)       # False (exact type check)
```

■ **NOTE:** Use isinstance() over type()==SomeType in your code. It correctly handles inheritance and is more Pythonic.

Input from User

The `input()` function reads a line of text from the user. It always returns a string — you must convert it if you need a number.

```
# Basic input
name = input('Enter your name: ')
print(f'Hello, {name}!')

# Numeric input – must convert
age = int(input('Enter your age: '))
gpa = float(input('Enter your GPA: '))

# Safe numeric input
def get_int(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print('Please enter a valid integer.')

age = get_int('Enter age: ')

# Multi-value input on one line
# Input: 10 20 30
a, b, c = map(int, input().split())
print(a, b, c)  # 10 20 30

# List from single line input
nums = list(map(int, input('Numbers: ').split()))
print(nums)  # [10, 20, 30]
```

■ INTERVIEW TIP:

In coding interviews, input is usually taken with `input().split()` and `map()`. Master this pattern for competitive programming and placement tests.

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Create variables of each type and print their types

```
name = 'Rohan'; age = 22; gpa = 9.1; passed = True
for v in [name, age, gpa, passed, None]:
    print(type(v).__name__, '→', v)
```

Exercise 2: Swap two numbers without a third variable

```
a, b = 10, 20
a, b = b, a
print(f'a={a}, b={b}') # a=20, b=10
```

Exercise 3: Take a temperature in Celsius and convert to Fahrenheit

```
celsius = float(input('Enter temp in Celsius: '))
fahrenheit = (celsius * 9/5) + 32
print(f'{celsius}°C = {fahrenheit:.1f}°F')
```

Exercise 4: Check if a value is truthy or falsy for 5 different values

```
for val in [0, '', [], None, 'hello', [1], 42, False]:
    print(f'bool({repr(val)}) = {bool(val)}')
```

Operators & Expressions

Performing calculations and comparisons in Python

What you will learn:

- Arithmetic operators

- Logical operators

- Assignment operators

- Operator precedence

- Comparison operators

- Bitwise operators

- Identity & membership

- Expressions & evaluation

Chapter 3 · [Complete Python Course](#)

Arithmetic & Assignment Operators

```
# Arithmetic
a, b = 15, 4
print(a + b)    # 19  addition
print(a - b)    # 11  subtraction
print(a * b)    # 60  multiplication
print(a / b)    # 3.75 true division (always float)
print(a // b)   # 3   floor division (integer result)
print(a % b)    # 3   modulo (remainder)
print(a ** b)   # 50625 power (15^4)
print(-a)      # -15 unary negation
```

Augmented Assignment (shorthand)

```
x = 100
x += 10    # x = x + 10 → 110
x -= 5     # x = x - 5  → 105
x *= 2     # x = x * 2  → 210
x //= 3    # x = x // 3 → 70
x %= 9     # x = x % 9  → 7
x **= 2    # x = x ** 2 → 49

# Walrus operator := (Python 3.8+)
# Assigns and returns in one step
import math
if (n := len([1,2,3,4,5])) > 3:
    print(f'List is long: {n} items') # 5
```

■ DIVISION NOTE:

In Python 3, / always returns a float ($5/2 = 2.5$). Use // for integer division ($5//2 = 2$). This is different from Python 2!

Comparison & Logical Operators

Comparison operators compare two values and return a boolean (True or False). Logical operators combine boolean expressions.

```
# Comparison operators
a, b = 10, 20
print(a == b)    # False   equal to
print(a != b)    # True    not equal to
print(a < b)     # True    less than
print(a > b)     # False   greater than
print(a <= b)    # True    less than or equal
print(a >= b)    # False   greater or equal

# Chained comparisons (Pythonic!)
x = 15
print(10 < x < 20)    # True — range check
print(1 < 2 < 3 < 4)  # True
```

Logical Operators

```
# and — True if BOTH are True
print(True and True)    # True
print(True and False)   # False

# or — True if AT LEAST ONE is True
print(False or True)    # True
print(False or False)   # False

# not — reverses the boolean
print(not True)         # False
print(not False)        # True

# Short-circuit evaluation
x = 5
# 'and' stops at first False
print(x > 0 and x < 10)  # True
# 'or' stops at first True
print(x > 10 or x < 3)   # False
```

Identity, Membership & Bitwise

Identity Operators

```
# 'is' checks same object in memory
a = [1, 2, 3]
b = a          # b points to same object
c = [1, 2, 3]  # c is a different object
print(a is b)  # True
print(a is c)  # False (different objects)
print(a == c)  # True (same value)
print(a is not c) # True

# Use 'is' for None checks
x = None
if x is None: print('empty') # correct
```

Membership Operators

```
# 'in' checks if value is in sequence
fruits = ['apple', 'banana', 'cherry']
print('apple' in fruits)    # True
print('grape' in fruits)    # False
print('grape' not in fruits) # True

# Works on strings too
msg = 'Hello Python'
print('Python' in msg)      # True
print('Java' in msg)        # False
```

Bitwise Operators

```
a, b = 0b1010, 0b1100 # 10 and 12
print(a & b)          # 0b1000 = 8   (AND)
print(a | b)          # 0b1110 = 14  (OR)
print(a ^ b)          # 0b0110 = 6   (XOR)
print(~a)             # -11         (NOT)
print(a << 1)         # 0b10100 = 20 (left shift)
print(a >> 1)         # 0b0101 = 5   (right shift)
```

Operator Precedence

When multiple operators appear in one expression, Python evaluates them in a fixed order (precedence). Higher precedence = evaluated first.

```
# Operator precedence (highest to lowest):
# 1. ()      - parentheses
# 2. **      - exponentiation
# 3. +x, -x  - unary plus/minus
# 4. * / // % - multiply, divide
# 5. + -     - add, subtract
# 6. << >>  - bitwise shifts
# 7. &       - bitwise AND
# 8. ^       - bitwise XOR
# 9. |       - bitwise OR
# 10. == != > >= < <= is in - comparisons
# 11. not    - logical NOT
# 12. and    - logical AND
# 13. or     - logical OR

# Examples
print(2 + 3 * 4)      # 14 (not 20!)
print((2 + 3) * 4)    # 20
print(2 ** 3 ** 2)    # 512 (right assoc: 3**2=9, 2**9=512)
print(10 - 4 - 2)     # 4 (left assoc: (10-4)-2=4)
```

■ BEST PRACTICE:

Use parentheses to make complex expressions clear, even when not strictly necessary. Readability beats cleverness every time.

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Calculate and print: area of a circle (radius=7)

```
import math
radius = 7
area = math.pi * radius ** 2
print(f'Area = {area:.2f}')
```

Exercise 2: Check if a year is a leap year (divisible by 4 but not 100, or by 400)

```
year = int(input('Enter year: '))
is_leap = (year%4==0 and year%100!=0) or (year%400==0)
print(f'{year} is{" " if is_leap else " not "}a leap year')
```

Exercise 3: Find if a number is even or odd using bitwise AND

```
n = int(input('Enter number: '))
if n & 1 == 0:
    print(f'{n} is even')
else:
    print(f'{n} is odd')
```

Strings

Python's most powerful built-in data type — mastered here

What you will learn:

■ String creation & literals

■ String methods

■ f-strings

■ Escape characters

■ Indexing & slicing

■ String formatting

■ String operations

■ String immutability

Chapter 4 · Complete Python Course

Creating Strings & Indexing

Strings in Python are sequences of Unicode characters, enclosed in single or double quotes. They are immutable — you cannot change a character in place.

```
# String creation
s1 = 'Hello'
s2 = "World"
s3 = '''Multi
line string'''
s4 = r'C:\Users\name' # raw string – no escape

# Indexing (0-based, negative from end)
s = 'Python'
#   P y t h o n
#   0 1 2 3 4 5   (positive)
#  -6 -5 -4 -3 -2 -1 (negative)
print(s[0])      # 'P'
print(s[-1])     # 'n'
print(s[2])      # 't'
print(s[-2])     # 'o'
```

String Slicing `s[start:stop:step]`

```
s = 'Hello, Python!'
print(s[0:5])      # 'Hello' (indices 0-4)
print(s[7:])       # 'Python!'
print(s[:5])       # 'Hello'
print(s[-7:])      # 'Python!'
print(s[::-2])     # 'Hlo yhn' (every 2nd char)
print(s[::-1])     # '!nohtyP ,olleH' (reverse!)
```

String Methods — Part 1

```
s = ' Hello, Python World! '
```



```
# Case methods
print(s.upper())      # ' HELLO, PYTHON WORLD! '
print(s.lower())      # ' hello, python world! '
print(s.title())      # ' Hello, Python World! '
print(s.capitalize()) # ' hello, ...' (first char only)
print(s.swapcase())   # swaps upper↔lower
```



```
# Stripping whitespace
print(s.strip())      # 'Hello, Python World!'
print(s.lstrip())     # removes leading spaces
print(s.rstrip())     # removes trailing spaces
```



```
# Searching
t = 'Hello, Python World!'
print(t.find('Python')) # 7 (index, -1 if not found)
print(t.index('Python')) # 7 (raises ValueError if not found)
print(t.count('l'))      # 3
print(t.startswith('He')) # True
print(t.endswith('!'))   # True
print('Python' in t)     # True
```

String Methods — Part 2

```
s = 'Hello, Python World!'

# Replace
print(s.replace('Python', 'Amazing')) # Hello, Amazing World!
print(s.replace('l', 'L', 2)) # replaces first 2 only

# Split & Join
words = s.split()           # split on whitespace
print(words)                # ['Hello,', 'Python', 'World!']
csv = '1,2,3,4,5'
nums = csv.split(',')       # ['1', '2', '3', '4', '5']

# join – opposite of split
joined = '-'.join(['a', 'b', 'c']) # 'a-b-c'
sentence = ' '.join(words)         # back to original

# Check content
print('123'.isdigit())    # True
print('abc'.isalpha())    # True
print('abc123'.isalnum()) # True
print(' '.isspace())      # True

# Padding & alignment
print('42'.zfill(5))      # '00042'
print('hi'.center(10, '*')) # '****hi****'
print('hi'.ljust(10, '-'))  # 'hi-----'
print('hi'.rjust(10, '-'))  # '-----hi'
```

String Formatting

1. % Formatting (old style)

```
name = 'Rohan'; age = 22; gpa = 9.12
print('Name: %s, Age: %d, GPA: %.2f' % (name, age, gpa))
# Name: Rohan, Age: 22, GPA: 9.12
```

2. .format() Method

```
print('Name: {}, Age: {}'.format(name, age))
print('{0} is {1} years old. {0} is a student.'.format(name, age))
print('GPA: {gpa:.2f}'.format(gpa=9.12))
```

3. f-strings (Python 3.6+ — PREFERRED)

```
# Basic f-string
print(f'Name: {name}, Age: {age}')

# Expressions inside f-strings
print(f'Next year I will be {age + 1}')
print(f'Square: {age ** 2}')

# Formatting numbers
pi = 3.14159265
print(f'Pi = {pi:.2f}')           # 3.14
print(f'Pi = {pi:.4f}')           # 3.1416
print(f'{1000000:,}')             # 1,000,000
print(f'{0.256:.1%}')             # 25.6%
print(f'{255:#x}')                # 0xff (hex)

# Alignment
print(f'{ "left":<10}|')          # 'left      |'
print(f'{ "right":>10}|')         # '          right|'
print(f'{ "centre":^10}|')        # '   centre   |'
```

■ USE F-STRINGS:

f-strings are the fastest, most readable way to format strings. Always prefer them over % or .format() for new code.

Escape Characters & Raw Strings

```
# Escape sequences
print('He said: \'Hello\'') # He said: 'Hello'
print("She said: \"Hi\"") # She said: "Hi"
print('Line1\nLine2')      # newline
print('Col1\tCol2')        # tab
print('Back\\slash')       # backslash
print('\a')                 # bell (alert)
print('\r')                 # carriage return

# Unicode escapes
print('\u03C0') # π (pi)
print('\U0001F600') # 🍌 emoji

# Raw strings – no escape processing
path1 = 'C:\\Users\\Rohan\\file.txt' # with escapes
path2 = r'C:\Users\Rohan\file.txt'   # raw string (cleaner)
print(path1 == path2)                # True – same result

# Regex often uses raw strings
import re
pattern = r'\d+\.\d+' # matches decimal numbers
match = re.search(pattern, 'Price: 19.99')
print(match.group())  # '19.99'
```

■ **NOTE:** Use raw strings (r'...') whenever your string contains backslashes — file paths, regex patterns, etc. It prevents accidental escape sequences.

String Operations & Immutability

```
# Concatenation
first = 'Hello'; second = ' World'
combined = first + second    # 'Hello World'

# Repetition
print('Ha' * 3)              # 'HaHaHa'
print('-' * 40)              # separator line

# Length
print(len('Python'))        # 6

# Membership
print('Py' in 'Python')     # True

# Iteration
for char in 'abc':
    print(char, end=' ')    # a b c

# Immutability
s = 'Hello'
# s[0] = 'h' → TypeError: strings are immutable!
s = s.replace('H', 'h')     # create a new string
# Or use list for mutability:
chars = list(s)
chars[0] = 'H'
s = ''.join(chars)
print(s)    # 'Hello'
```

Useful Built-in String Functions

```
s = 'Python Programming'
print(len(s))              # 18
print(min(s))              # ' ' (space has lowest ASCII)
print(max(s))              # 't' (highest ASCII here)
print(sorted(s))           # sorted list of chars
print(list(enumerate(s[:5])))
# [(0, 'P'), (1, 'y'), (2, 't'), (3, 'h'), (4, 'o')]
```

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Count vowels in a given string

```
def count_vowels(s):  
    return sum(1 for c in s.lower() if c in 'aeiou')  
print(count_vowels('Hello Python')) # 4
```

Exercise 2: Reverse words in a sentence

```
sentence = 'Python is awesome'  
reversed_words = ' '.join(sentence.split()[::-1])  
print(reversed_words) # 'awesome is Python'
```

Exercise 3: Check if two strings are anagrams

```
def is_anagram(s1, s2):  
    return sorted(s1.lower()) == sorted(s2.lower())  
print(is_anagram('listen', 'silent')) # True
```

Exercise 4: Format a table of students with names and grades

```
students = [('Alice', 95), ('Bob', 87), ('Carol', 92)]  
print(f'{{Name':<10}} {{Grade':>6}}')  
print('-'*18)  
for name, grade in students:  
    print(f'{{name':<10}} {{grade':>6}}')
```

Control Flow

Making decisions in Python with if, elif, and else

What you will learn:

■ if statement

■ if-elif-else chains

■ Ternary operator

■ Boolean logic in conditions

■ if-else

■ Nested if

■ match-case (Python 3.10+)

■ Practical patterns

Chapter 5 · Complete Python Course

The if Statement

The if statement executes a block of code only if the condition is True. Python uses indentation (4 spaces) to define the block — no curly braces needed.

```
# Basic if
age = 18
if age >= 18:
    print('You are an adult')
    print('You can vote')
print('This always runs')

# if-else
score = 75
if score >= 50:
    print('PASS')
else:
    print('FAIL')

# if-elif-else
marks = 82
if marks >= 90:
    grade = 'A+'
elif marks >= 80:
    grade = 'A'
elif marks >= 70:
    grade = 'B'
elif marks >= 60:
    grade = 'C'
else:
    grade = 'F'
print(f'Grade: {grade}')    # Grade: A
```

■ IMPORTANT:

Python checks elif conditions in order, top to bottom. As soon as one is True, the rest are skipped.

Nested if & Ternary Operator

```
# Nested if
age      = 20
has_id   = True

if age >= 18:
    if has_id:
        print('Entry allowed')
    else:
        print('Need ID')
else:
    print('Too young')

# Same thing – cleaner with 'and'
if age >= 18 and has_id:
    print('Entry allowed')
```

Ternary (Conditional) Expression

```
# Syntax: value_if_true if condition else value_if_false

x = 10
result = 'positive' if x > 0 else 'non-positive'
print(result)    # 'positive'

# Use in assignments
score = 72
status = 'PASS' if score >= 50 else 'FAIL'
print(status)    # 'PASS'

# Ternary with function calls
name = input('Name: ')
greeting = f'Hello, {name}' if name else 'Hello, stranger'

# Nested ternary (use sparingly – unreadable!)
n = 0
label = 'positive' if n>0 else 'negative' if n<0 else 'zero'
print(label)     # 'zero'
```

Match-Case (Python 3.10+)

The match-case statement is Python's structural pattern matching, introduced in Python 3.10. It is more powerful than a switch statement in other languages.

```
# Basic match-case (like switch in other languages)
command = 'quit'

match command:
    case 'quit':
        print('Exiting...')
    case 'help':
        print('Available commands...')
    case 'start':
        print('Starting...')
    case _:
        # default case
        print(f'Unknown: {command}')

# Matching with conditions (guards)
score = 87
match score:
    case s if s >= 90: print('A+')
    case s if s >= 80: print('A')
    case s if s >= 70: print('B')
    case _:
        print('C or below')

# Matching tuples/structures
point = (0, 5)
match point:
    case (0, 0): print('Origin')
    case (0, y): print(f'Y-axis at {y}')
    case (x, 0): print(f'X-axis at {x}')
    case (x, y): print(f'Point ({x},{y})')
```

Practical if Patterns

Common Patterns

```
# 1. Validate input
age = int(input('Enter age: '))
if not (0 < age < 150):
    print('Invalid age!')
else:
    print(f'Valid age: {age}')

# 2. Check membership
role = 'admin'
allowed = ['admin', 'moderator']
if role in allowed:
    print('Access granted')

# 3. Check None before using
result = None
if result is not None:
    print(result.upper())

# 4. Short-circuit tricks
name = input('Name: ')
name = name or 'Anonymous' # default if empty

# 5. One-liner if for simple assignments
x = 5
if x > 0: sign = 1
elif x < 0: sign = -1
else: sign = 0

# 6. FizzBuzz classic
for n in range(1, 21):
    if n%15==0: print('FizzBuzz')
    elif n%3==0: print('Fizz')
    elif n%5==0: print('Buzz')
    else:      print(n)
```

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Write a BMI calculator with category labels

```
weight = float(input('Weight (kg): '))
height = float(input('Height (m): '))
bmi = weight / height**2
if bmi < 18.5:    cat='Underweight'
elif bmi < 25:   cat='Normal'
elif bmi < 30:   cat='Overweight'
else:            cat='Obese'
print(f'BMI: {bmi:.1f} — {cat}')
```

Exercise 2: Determine if a triangle is valid given 3 sides

```
a,b,c = map(float, input('3 sides: ').split())
if a+b>c and b+c>a and a+c>b:
    print('Valid triangle')
else:
    print('Not a triangle')
```

Exercise 3: Rock Paper Scissors — one round

```
import random
choices = ['rock', 'paper', 'scissors']
user = input('rock/paper/scissors: ').lower()
comp = random.choice(choices)
print(f'Computer chose: {comp}')
if user==comp: print('Draw!')
elif (user=='rock' and comp=='scissors') or \
     (user=='paper' and comp=='rock') or \
     (user=='scissors' and comp=='paper'):
    print('You win!')
else: print('Computer wins!')
```

Loops

Repeating actions with for and while loops

What you will learn:

- for loop

- range()

- pass

- Nested loops

- while loop

- break & continue

- Loop else

- enumerate() & zip()

Chapter 6 · Complete Python Course

The for Loop

The for loop iterates over any iterable — lists, strings, tuples, ranges, files, generators. Python's for loop is more like a 'for-each' loop in other languages.

```
# Iterate over a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Iterate over a string
for char in 'Python':
    print(char, end=' ') # P y t h o n

# range(stop)          - 0 to stop-1
for i in range(5):
    print(i, end=' ') # 0 1 2 3 4

# range(start, stop)   - start to stop-1
for i in range(2, 7):
    print(i, end=' ') # 2 3 4 5 6

# range(start, stop, step)
for i in range(0, 20, 5):
    print(i, end=' ') # 0 5 10 15

# Countdown - negative step
for i in range(5, 0, -1):
    print(i, end=' ') # 5 4 3 2 1
```

enumerate(), zip() & range()

enumerate() — loop with index

```
fruits = ['apple', 'banana', 'cherry']

# Without enumerate (awkward)
for i in range(len(fruits)):
    print(i, fruits[i])

# With enumerate (Pythonic!)
for i, fruit in enumerate(fruits):
    print(i, fruit)

# Custom start index
for i, fruit in enumerate(fruits, start=1):
    print(f'{i}. {fruit}')
# 1. apple 2. banana 3. cherry
```

zip() — iterate over multiple lists

```
names = ['Alice', 'Bob', 'Carol']
scores = [95, 87, 92]

for name, score in zip(names, scores):
    print(f'{name}: {score}')
# Alice: 95 Bob: 87 Carol: 92

# zip stops at shortest list
# Use itertools.zip_longest for longest

# Unzip with zip(*)
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
nums, chars = zip(*pairs)
print(nums) # (1, 2, 3)
print(chars) # ('a', 'b', 'c')
```

The while Loop

The while loop repeats a block as long as the condition is True. Use it when you do not know the number of iterations in advance.

```
# Basic while
count = 0
while count < 5:
    print(count, end=' ') # 0 1 2 3 4
    count += 1

# Input validation loop
while True:
    age = int(input('Age (1-120): '))
    if 1 <= age <= 120:
        break
    print('Invalid! Try again.')
print(f'Age accepted: {age}')
```

```
# Countdown timer
n = 5
while n > 0:
    print(n)
    n -= 1
print('Liftoff!')
```

```
# do-while equivalent
while True:
    action = input('Enter command: ')
    process(action) # do something
    if action == 'quit': break
```

■■ INFINITE LOOP:

Always make sure your while loop's condition will eventually become False, or that you have a break statement. An infinite loop hangs your program.

break, continue, pass & Loop else

```
# break – exit the loop immediately
for n in range(10):
    if n == 5:
        break
    print(n, end=' ') # 0 1 2 3 4

# continue – skip rest of iteration
for n in range(10):
    if n % 2 == 0:
        continue # skip evens
    print(n, end=' ') # 1 3 5 7 9

# pass – do nothing (placeholder)
for n in range(5):
    if n == 3:
        pass # do nothing for 3
    print(n, end=' ') # 0 1 2 3 4 (all printed)

# Loop else – runs if loop completes normally (no break)
for n in range(2, 10):
    for factor in range(2, n):
        if n % factor == 0:
            break # not prime
    else:
        print(n, end=' ') # 2 3 5 7 (primes!)

# while else
i = 0
while i < 5:
    i += 1
else:
    print('Loop finished normally')
```

Nested Loops & Patterns

```
# Nested for loops
for i in range(1, 4):
    for j in range(1, 4):
        print(f'{i}x{j}={i*j}', end=' ')
    print() # newline after each row

# Triangle pattern
n = 5
for i in range(1, n+1):
    print('*' * i)
# *  **  ***  ****  *****

# Multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f'{i*j:3}', end='')
    print()

# List of lists – nested loops
matrix = [[1,2,3],[4,5,6],[7,8,9]]
for row in matrix:
    for val in row:
        print(val, end=' ')
    print()
```

List Comprehension vs Loop

```
# Loop version
squares = []
for x in range(10):
    squares.append(x**2)

# Comprehension – same result, one line
squares = [x**2 for x in range(10)]
print(squares) # [0,1,4,9,16,25,36,49,64,81]
```

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Print multiplication table for a given number

```
n = int(input('Table for: '))
for i in range(1, 11):
    print(f'{n} x {i:2} = {n*i:3}')
```

Exercise 2: Number guessing game (use while loop)

```
import random
secret = random.randint(1, 100)
attempts = 0
while True:
    guess = int(input('Guess (1-100): '))
    attempts += 1
    if guess < secret: print('Too low!')
    elif guess > secret: print('Too high!')
    else:
        print(f'Correct! {attempts} attempts')
        break
```

Exercise 3: Print all prime numbers up to N

```
n = int(input('Find primes up to: '))
for num in range(2, n+1):
    for i in range(2, int(num**0.5)+1):
        if num % i == 0: break
    else:
        print(num, end=' ')
```

Functions

Writing reusable, modular, and clean Python code

What you will learn:

■ Defining functions

■ Default arguments

■ Return values

■ Lambda functions

■ Docstrings

■ Parameters & arguments

■ *args & **kwargs

■ Scope (LEGB rule)

■ Recursion

■ Higher-order functions

Chapter 7 · Complete Python Course

Defining & Calling Functions

A function is a named, reusable block of code. Functions reduce repetition (DRY principle) and make code modular and testable.

```
# Define a function
def greet(name):
    '''Greet a user by name.''' # docstring
    message = f'Hello, {name}!'
    print(message)

# Call the function
greet('Rohan')      # Hello, Rohan!
greet('Alice')      # Hello, Alice!

# Function with return value
def add(a, b):
    return a + b

result = add(10, 20)
print(result)       # 30

# Multiple return values (returns a tuple)
def min_max(lst):
    return min(lst), max(lst)

lo, hi = min_max([3, 1, 4, 1, 5, 9])
print(lo, hi)       # 1 9

# Returning None implicitly
def do_nothing():
    pass
print(do_nothing()) # None
```

■ DRY PRINCIPLE:

Don't Repeat Yourself. If you write the same code more than twice, put it in a function. Functions make code maintainable.

Parameters & Arguments

```
# Positional arguments – order matters
def describe(name, age, city):
    print(f'{name}, {age}, from {city}')

describe('Rohan', 22, 'Pune') # positional

# Keyword arguments – order doesn't matter
describe(age=22, city='Pune', name='Rohan')

# Default arguments
def greet(name, greeting='Hello'):
    print(f'{greeting}, {name}!')

greet('Rohan') # Hello, Rohan!
greet('Alice', 'Hi') # Hi, Alice!
greet('Bob', greeting='Hey') # Hey, Bob!
```

*args — Variable Positional Arguments

```
def total(*args):
    return sum(args)

print(total(1, 2, 3)) # 6
print(total(10, 20, 30, 40)) # 100
```

**kwargs — Variable Keyword Arguments

```
def show_info(**kwargs):
    for key, val in kwargs.items():
        print(f' {key}: {val}')

show_info(name='Rohan', age=22, city='Pune')

# Combining all parameter types
def f(a, b, *args, **kwargs):
    print(a, b, args, kwargs)

f(1, 2, 3, 4, x=5, y=6)
# 1 2 (3, 4) {'x': 5, 'y': 6}
```

Scope — LEGB Rule

When you use a variable name, Python searches for it in this order: Local → Enclosing → Global → Built-in. This is called the LEGB rule.

```
x = 'global'    # Global scope

def outer():
    x = 'enclosing'    # Enclosing scope

    def inner():
        x = 'local'    # Local scope
        print(x)       # 'local' — uses local first

    inner()
    print(x)           # 'enclosing'

outer()
print(x)              # 'global'

# global keyword — modify global from inside function
count = 0
def increment():
    global count
    count += 1

increment(); increment()
print(count)          # 2

# nonlocal — modify enclosing scope
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc

c = counter()
print(c(), c(), c())  # 1 2 3
```

Lambda Functions

Lambda creates an anonymous function in a single line. It is a function without a name, used for short operations — typically with `map()`, `filter()`, `sorted()`.

```
# Syntax: lambda arguments: expression

# Regular function
def square(x):
    return x ** 2

# Lambda equivalent
square = lambda x: x ** 2
print(square(5))      # 25

# Lambda with multiple args
add = lambda a, b: a + b
print(add(3, 4))      # 7

# Lambda with conditional
sign = lambda x: 1 if x>0 else -1 if x<0 else 0
print(sign(-5))       # -1

# sorted() with lambda key
students = [('Alice', 92), ('Bob', 78), ('Carol', 95)]
by_score = sorted(students, key=lambda s: s[1], reverse=True)
print(by_score)
# [('Carol', 95), ('Alice', 92), ('Bob', 78)]

# filter() with lambda
nums = range(1, 11)
evens = list(filter(lambda x: x%2==0, nums))
print(evens) # [2,4,6,8,10]

# map() with lambda
doubled = list(map(lambda x: x*2, [1,2,3,4,5]))
print(doubled) # [2,4,6,8,10]
```

Recursion

Recursion is when a function calls itself. Every recursive function needs: a base case (stops the recursion) and a recursive case (calls itself with a simpler input).

```
# Factorial – classic recursion example
def factorial(n):
    if n == 0 or n == 1:    # base case
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(5))    # 120  (5*4*3*2*1)
print(factorial(10))   # 3628800

# Fibonacci sequence
def fib(n):
    if n <= 1: return n      # base case
    return fib(n-1) + fib(n-2) # recursive

for i in range(8):
    print(fib(i), end=' ') # 0 1 1 2 3 5 8 13

# Efficient Fibonacci with memoization
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_fast(n):
    if n <= 1: return n
    return fib_fast(n-1) + fib_fast(n-2)

print(fib_fast(50))    # 12586269025 (fast!)
```

■■ RECURSION LIMIT:

Python has a default recursion limit of 1000. Use `sys.setrecursionlimit()` to change it. For deep recursion, use iteration or memoization.

Docstrings & Function Best Practices

```
def calculate_bmi(weight_kg, height_m):  
    '''  
    Calculate Body Mass Index (BMI).  
  
    Args:  
        weight_kg (float): Weight in kilograms.  
        height_m (float): Height in metres.  
  
    Returns:  
        float: BMI value rounded to 2 decimal places.  
  
    Raises:  
        ValueError: If height is zero or negative.  
  
    Example:  
        >>> calculate_bmi(70, 1.75)  
        22.86  
    '''  
    if height_m <= 0:  
        raise ValueError('Height must be positive')  
    return round(weight_kg / height_m**2, 2)  
  
# Access docstring  
print(calculate_bmi.__doc__)  
help(calculate_bmi)  
  
# Test it  
print(calculate_bmi(70, 1.75))  # 22.86
```

Function Best Practices

- One function = one responsibility (Single Responsibility Principle)
- Name functions as verbs: `get_score()`, `calculate_bmi()`, `send_email()`
- Keep functions short — ideally under 20 lines
- Validate inputs early — fail fast with clear error messages
- Always write a docstring for functions others will use

Practice Exercises

—■ Write the code yourself before checking the solution. Practice = mastery.

Exercise 1: Write a function to check if a number is perfect

```
def is_perfect(n):
    return n == sum(i for i in range(1,n) if n%i==0)
print(is_perfect(6))    # True   (1+2+3=6)
print(is_perfect(28))   # True
print(is_perfect(12))   # False
```

Exercise 2: Fibonacci using memoization (functools.lru_cache)

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
print([fib(i) for i in range(10)])
```

Exercise 3: Create a calculator using a dict of lambda functions

```
ops = {
    '+': lambda a,b: a+b,
    '-': lambda a,b: a-b,
    '*': lambda a,b: a*b,
    '/': lambda a,b: a/b if b else 'Error'
}
a,op,b = input('e.g. 10 + 5: ').split()
print(ops[op](float(a), float(b)))
```

Lists

Python's most versatile built-in data structure

What you will learn:

■ Creating lists

■ List methods

■ List comprehension

■ Sorting

■ Accessing elements

■ Slicing

■ Nested lists

■ Common patterns

Chapter 8 · Complete Python Course

Creating & Accessing Lists

A list is an ordered, mutable collection that can hold any mix of data types. Lists are defined with square brackets `[]` and support indexing, slicing, and iteration.

```
# Creating lists
empty = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, 'hello', 3.14, True, None]
nested = [[1,2],[3,4],[5,6]]

# list() constructor
chars = list('Python')    # ['P','y','t','h','o','n']
zeros = [0] * 5           # [0,0,0,0,0]
seq = list(range(5))      # [0,1,2,3,4]

# Indexing
fruits = ['apple','banana','cherry','date']
print(fruits[0])          # 'apple'
print(fruits[-1])         # 'date'
print(fruits[1:3])        # ['banana','cherry']
print(fruits[:2])         # ['apple','cherry']
print(fruits[::-1])       # reversed list

# Modify elements
fruits[0] = 'avocado'
print(fruits)             # ['avocado','banana','cherry','date']
```

List Methods

```
lst = [3, 1, 4, 1, 5, 9, 2, 6]

# Adding elements
lst.append(7)          # add to end: [3,1,4,...,7]
lst.insert(0, 0)       # insert at index 0
lst.extend([10, 11])   # add multiple at end

# Removing elements
lst.remove(1)          # removes FIRST occurrence of 1
popped = lst.pop()     # removes & returns last element
popped = lst.pop(0)    # removes & returns at index 0
del lst[0]             # delete by index
lst.clear()            # remove all elements

# Searching
nums = [10, 20, 30, 20, 40]
print(nums.index(20))  # 1 (first occurrence)
print(nums.count(20))  # 2
print(20 in nums)     # True

# Sorting
nums.sort()            # in-place ascending
nums.sort(reverse=True) # in-place descending
sorted_nums = sorted(nums) # new sorted list

# Other
nums.reverse()         # reverse in place
copy = nums.copy()     # shallow copy
print(len(nums))       # length
```

List Comprehension

List comprehension is a concise, Pythonic way to create lists. It is faster than a for loop because it is implemented in C internally.

```
# Syntax: [expression for item in iterable if condition]
```

```
# Basic – squares of 0-9
```

```
squares = [x**2 for x in range(10)]
```

```
# [0,1,4,9,16,25,36,49,64,81]
```

```
# With condition – even numbers only
```

```
evens = [x for x in range(20) if x%2 == 0]
```

```
# [0,2,4,6,8,10,12,14,16,18]
```

```
# Transform strings
```

```
words = ['hello','world','python']
```

```
upper = [w.upper() for w in words]
```

```
# ['HELLO','WORLD','PYTHON']
```

```
# Nested comprehension – flatten 2D list
```

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
```

```
flat = [x for row in matrix for x in row]
```

```
# [1,2,3,4,5,6,7,8,9]
```

```
# Conditional expression inside
```

```
labeled = ['even' if x%2==0 else 'odd' for x in range(6)]
```

```
# ['even','odd','even','odd','even','odd']
```

```
# Filter and transform
```

```
nums = [3,-1,4,-1,5,-9,2,-6]
```

```
pos_sq = [x**2 for x in nums if x > 0]
```

```
# [9,16,25,4]
```

■ SPEED:

List comprehensions are typically 30-50% faster than equivalent for loops for simple transformations. Prefer them for clean, readable code.

Sorting & Advanced List Operations

```
# sorted() – returns new list, original unchanged
nums = [3, 1, 4, 1, 5, 9]
print(sorted(nums))           # [1,1,3,4,5,9]
print(sorted(nums, reverse=True)) # [9,5,4,3,1,1]
print(nums)                   # [3,1,4,1,5,9] unchanged

# Sort by key
words = ['banana', 'apple', 'cherry', 'fig']
print(sorted(words))          # alphabetical
print(sorted(words, key=len)) # by length
print(sorted(words, key=lambda w: w[-1])) # by last char

# Sort list of dicts
students = [
    {'name': 'Rohan', 'gpa': 9.1},
    {'name': 'Alice', 'gpa': 9.5},
    {'name': 'Bob', 'gpa': 8.7},
]
by_gpa = sorted(students, key=lambda s: s['gpa'], reverse=True)

# zip, enumerate, map on lists
a = [1,2,3]; b = [4,5,6]
pairs = list(zip(a, b))      # [(1,4),(2,5),(3,6)]
sums = list(map(sum, pairs)) # [5,7,9]
total = sum(a)               # 6
product = 1
for x in a: product *= x     # 6
print(all(x>0 for x in a))   # True
print(any(x>2 for x in a))   # True
```

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Remove duplicates from a list while preserving order

```
def remove_dups(lst):
    seen = set(); result = []
    for x in lst:
        if x not in seen:
            seen.add(x); result.append(x)
    return result
print(remove_dups([1,2,2,3,1,4,3])) # [1,2,3,4]
```

Exercise 2: Rotate a list by k positions to the right

```
def rotate(lst, k):
    k %= len(lst)
    return lst[-k:] + lst[:-k]
print(rotate([1,2,3,4,5], 2)) # [4,5,1,2,3]
```

Exercise 3: Find the second largest element without sorting

```
def second_largest(lst):
    first = second = float('-inf')
    for x in lst:
        if x > first: second, first = first, x
        elif x > second and x != first: second = x
    return second
print(second_largest([3,1,4,1,5,9,2,6])) # 6
```

Tuples & Sets

Immutable sequences and unique collections

What you will learn:

- Creating tuples

- Named tuples

- Set operations

- When to use tuple vs list

- Tuple unpacking

- Creating sets

- frozenset

- Set comprehension

Chapter 9 · Complete Python Course

Tuples — Immutable Sequences

A tuple is an ordered, immutable collection. Once created, you cannot change it. Tuples are faster than lists and are used for fixed data like coordinates, RGB colours, database rows.

```
# Creating tuples
empty = ()                # empty tuple
single = (42,)            # single item – note the comma!
point = (3, 4)            # 2D point
rgb = (255, 128, 0)       # colour
mixed = (1, 'hello', 3.14) # mixed types

# Without parentheses (packing)
t = 1, 2, 3               # same as (1, 2, 3)

# Accessing
print(point[0])           # 3
print(point[-1])          # 4
print(rgb[1:])            # (128, 0)

# Tuple unpacking
x, y = point
r, g, b = rgb
print(x, y)               # 3 4

# Extended unpacking with *
first, *rest = (1, 2, 3, 4, 5)
print(first)              # 1
print(rest)               # [2, 3, 4, 5]

# Tuple methods (only 2!)
t = (1, 2, 2, 3, 2)
print(t.count(2))         # 3
print(t.index(3))         # 3
```

■ **NOTE:** Tuples are hashable (if all elements are hashable) — they can be used as dictionary keys and set members, unlike lists.

Sets — Unique Collections

A set is an unordered collection of unique elements. Sets support fast membership testing $O(1)$ and mathematical set operations.

```
# Creating sets
empty = set()           # NOT {} – that's an empty dict!
nums = {1, 2, 3, 4, 5}
auto = {1, 1, 2, 2, 3}  # auto-dedup → {1, 2, 3}
from_list = set([1,2,2,3]) # {1, 2, 3}

# Set operations
a = {1, 2, 3, 4, 5}
b = {3, 4, 5, 6, 7}

print(a | b)    # Union:      {1,2,3,4,5,6,7}
print(a & b)    # Intersection: {3,4,5}
print(a - b)    # Difference:  {1,2}
print(b - a)    # Difference:  {6,7}
print(a ^ b)    # Symmetric diff: {1,2,6,7}

# Methods
s = {1, 2, 3}
s.add(4)        # add one element
s.update([5,6]) # add multiple
s.remove(1)     # remove (raises KeyError if missing)
s.discard(99)   # remove (no error if missing)
popped = s.pop() # remove and return arbitrary element

# Subsets
print({1,2} <= {1,2,3}) # True – subset
print({1,2} < {1,2,3})  # True – proper subset
```

■ PERFORMANCE:

Membership test in set: $O(1)$. In list: $O(n)$. For large collections, converting to a set for lookups is a huge speed win.

Set Comprehension & frozenset

```
# Set comprehension
squares = {x**2 for x in range(10)}
# {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}

# Remove duplicates from a string
unique_chars = {c for c in 'mississippi'}
# {'m', 'i', 's', 'p'}

# Find common elements (readable)
list1 = [1,2,3,4,5,2,3]
list2 = [3,4,5,6,7,4,5]
common = set(list1) & set(list2)
print(common)    # {3, 4, 5}

# frozenset – immutable, hashable set
fs = frozenset([1, 2, 3])
# Can be used as dict key
d = {frozenset([1,2]): 'pair'}
print(d[frozenset([2,1])])  # 'pair' (order doesn't matter)

# Use case: find all unique pairs
pairs = [(1,2),(2,1),(3,4)]
unique = {frozenset(p) for p in pairs}
print(len(unique))  # 2 (not 3)
```

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Find common elements in two lists without duplicates

```
a = [1,2,2,3,4,4,5]; b = [3,4,5,6,7]
common = list(set(a) & set(b))
print(sorted(common)) # [3, 4, 5]
```

Exercise 2: Use tuple unpacking to swap variables in a list

```
data = [('Alice',92),('Bob',85),('Carol',91)]
for name, score in data:
    print(f'{name}: {score}/100')
```

Exercise 3: Remove duplicates using a set but keep original order

```
def unique_ordered(lst):
    seen = set()
    return [x for x in lst if not (x in seen or seen.add(x))]
print(unique_ordered([3,1,4,1,5,9,2,6,5,3])) # [3,1,4,5,9,2,6]
```

Dictionaries

Key-value mappings — Python's most powerful built-in

What you will learn:

■ Creating dicts

■ Dict methods

■ Dict comprehension

■ defaultdict & Counter

■ Accessing & updating

■ Iteration

■ Nested dicts

■ Merging dicts

Chapter 10 · Complete Python Course

Creating & Accessing Dictionaries

A dictionary is an ordered (Python 3.7+), mutable collection of key-value pairs. Keys must be unique and immutable (str, int, tuple). Values can be anything.

```
# Creating
empty = {}
person = {'name': 'Rohan', 'age': 22, 'city': 'Pune'}
from_pairs = dict([('a', 1), ('b', 2)])
from_keys = dict.fromkeys(['x', 'y', 'z'], 0)
# {'x': 0, 'y': 0, 'z': 0}

# Accessing
print(person['name'])          # 'Rohan' (KeyError if missing!)
print(person.get('age'))       # 22
print(person.get('phone', 'N/A')) # 'N/A' (default if missing)

# Modifying
person['age'] = 23              # update
person['email'] = 'r@mail.com'  # add new key
del person['city']              # delete key
removed = person.pop('age')     # remove + return value
person.update({'age': 23, 'phone': '999'}) # bulk update

# Check keys
print('name' in person)        # True
print('phone' not in person)   # True/False
print(len(person))             # number of keys
```

Dict Methods & Iteration

```
d = {'a':1,'b':2,'c':3}

# Keys, values, items
print(list(d.keys()))    # ['a','b','c']
print(list(d.values()))  # [1, 2, 3]
print(list(d.items()))   # [('a',1),('b',2),('c',3)]

# Iteration
for key in d:
    print(key, d[key])

for key, val in d.items():
    print(f'{key}: {val}')

#.setdefault
d.setdefault('d', 0)    # adds 'd':0 only if 'd' absent
print(d)                # {'a':1,'b':2,'c':3,'d':0}

# Merge dicts
d1 = {'a':1,'b':2}; d2 = {'b':3,'c':4}
# Python 3.9+ merge operator
merged = d1 | d2         # {'a':1,'b':3,'c':4}
d1 |= d2                 # update in place

# Pre-3.9 merge
merged = {**d1, **d2}    # also works in 3.5+
```

Dict Comprehension

```
# {key_expr: val_expr for item in iterable if condition}
squares = {x: x**2 for x in range(6)}
# {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

# Swap keys and values
original = {'a':1,'b':2,'c':3}
swapped = {v:k for k,v in original.items()}
# {1:'a', 2:'b', 3:'c'}

# Filter items
scores = {'Alice':92,'Bob':65,'Carol':88,'Dave':55}
passed = {k:v for k,v in scores.items() if v >= 70}
# {'Alice':92,'Carol':88}
```

defaultdict, Counter & OrderedDict

```
from collections import defaultdict, Counter, OrderedDict

# defaultdict – provides default for missing keys
word_count = defaultdict(int)
for word in 'the quick brown fox the fox'.split():
    word_count[word] += 1
print(dict(word_count))
# {'the':2, 'quick':1, 'brown':1, 'fox':2}

# Group items
from_city = defaultdict(list)
students = [('Alice', 'Pune'), ('Bob', 'Mumbai'), ('Carol', 'Pune')]
for name, city in students:
    from_city[city].append(name)
print(dict(from_city))
# {'Pune': ['Alice', 'Carol'], 'Mumbai': ['Bob']}

# Counter – count hashable objects
c = Counter('banana')
print(c)                # Counter({'a':3, 'n':2, 'b':1})
print(c.most_common(2)) # [('a', 3), ('n', 2)]
print(c['a'])            # 3
print(c['z'])            # 0 (no KeyError!)

# Counter arithmetic
c1 = Counter(['a', 'b', 'a'])
c2 = Counter(['a', 'c'])
print(c1 + c2)           # Counter({'a':3, 'b':1, 'c':1})
print(c1 - c2)           # Counter({'a':1, 'b':1})
```

■ INTERVIEW:

Counter is used in 60%+ of string manipulation interview problems. Know: Counter(s).most_common(n), arithmetic, and iteration.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Count frequency of each character in a string

```
from collections import Counter
s = input('String: ')
freq = Counter(s)
for char, count in sorted(freq.items()):
    print(f' {repr(char)}: {count}')
```

Exercise 2: Group a list of words by their first letter

```
from collections import defaultdict
words = ['apple', 'ant', 'banana', 'bear', 'cat', 'cherry']
grouped = defaultdict(list)
for w in words: grouped[w[0]].append(w)
for k,v in sorted(grouped.items()):
    print(f'{k}: {v}')
```

Exercise 3: Invert a dictionary (swap keys and values)

```
original = {'a':1, 'b':2, 'c':3}
inverted = {v:k for k,v in original.items()}
print(inverted) # {1:'a', 2:'b', 3:'c'}
```

File Handling

Reading, writing, and managing files with Python

What you will learn:

- `open()` and modes

- Writing files

- File methods

- JSON files

- Reading files

- `with` statement

- CSV files

- `os` & `pathlib` modules

[Chapter 11](#) · [Complete Python Course](#)

Opening Files & File Modes

Python's built-in `open()` function is used to work with files. Always use the `'with'` statement — it ensures the file is closed even if an error occurs.

```
# File modes:
# 'r' - read (default)      - file must exist
# 'w' - write              - creates/overwrites
# 'a' - append            - creates/adds to end
# 'x' - exclusive create   - fails if file exists
# 'r+' - read + write      - file must exist
# 'b' - binary mode (rb, wb, ab)
# 't' - text mode (default)

# Always use 'with' - auto-closes the file
with open('example.txt', 'w') as f:
    f.write('Hello, File World!\n')
    f.write('Line 2\n')
# File is automatically closed here

# Read entire file
with open('example.txt', 'r') as f:
    content = f.read()
    print(content)

# Specify encoding (always do this!)
with open('data.txt', 'r', encoding='utf-8') as f:
    text = f.read()
```

■ ALWAYS SPECIFY ENCODING:

Omitting encoding causes issues on different operating systems. Always use `encoding='utf-8'` when working with text files.

Reading & Writing Files

```
# Writing
lines = ['Python\n', 'is\n', 'awesome\n']
with open('lines.txt', 'w') as f:
    f.writelines(lines)    # write list of strings

# Reading methods
with open('lines.txt', 'r') as f:
    # read() – entire file as one string
    content = f.read()

with open('lines.txt', 'r') as f:
    # readline() – one line at a time
    line1 = f.readline()  # 'Python\n'
    line2 = f.readline()  # 'is\n'

with open('lines.txt', 'r') as f:
    # readlines() – list of all lines
    all_lines = f.readlines()
    # ['Python\n', 'is\n', 'awesome\n']

# Best way – iterate line by line (memory efficient)
with open('lines.txt', 'r') as f:
    for line in f:
        print(line.strip()) # strip removes \n

# Append to existing file
with open('lines.txt', 'a') as f:
    f.write('...and fast!\n')
```

CSV & JSON File Handling

CSV Files

```
import csv

# Write CSV
with open('students.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Name', 'Age', 'GPA']) # header
    writer.writerow(['Alice', 20, 9.5])
    writer.writerow(['Bob', 22, 8.7])

# Write CSV with DictWriter
with open('students.csv', 'w', newline='') as f:
    fields = ['Name', 'Age', 'GPA']
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerow({'Name': 'Alice', 'Age': 20, 'GPA': 9.5})

# Read CSV
with open('students.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['GPA'])
```

JSON Files

```
import json

data = {'name': 'Rohan', 'scores': [90, 85, 92], 'active': True}

# Write JSON
with open('data.json', 'w') as f:
    json.dump(data, f, indent=2)

# Read JSON
with open('data.json') as f:
    loaded = json.load(f)
    print(loaded['name']) # Rohan

# JSON string conversion
s = json.dumps(data)      # dict → JSON string
d = json.loads(s)         # JSON string → dict
```

os and pathlib Modules

```
import os
from pathlib import Path

# os module
print(os.getcwd())           # current directory
os.mkdir('new_folder')       # create directory
os.makedirs('a/b/c')         # create nested dirs
os.rename('old.txt', 'new.txt') # rename file
os.remove('file.txt')        # delete file
os.rmdir('empty_folder')     # delete empty dir

# List directory
files = os.listdir('.')      # list files
for f in os.scandir('.'):
    print(f.name, f.is_file())

# os.path
path = '/home/user/docs/file.txt'
print(os.path.dirname(path))  # '/home/user/docs'
print(os.path.basename(path)) # 'file.txt'
print(os.path.exists(path))   # True/False
print(os.path.getsize(path))  # file size in bytes

# pathlib – modern, OOP way (preferred)
p = Path('data/students.csv')
print(p.name)                 # 'students.csv'
print(p.stem)                 # 'students'
print(p.suffix)               # '.csv'
print(p.parent)               # 'data'
print(p.exists())             # True/False
p.mkdir(parents=True, exist_ok=True)
text = p.read_text(encoding='utf-8')
p.write_text('hello', encoding='utf-8')
```

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Write a word-count program that reads a file

```
from collections import Counter
with open('sample.txt', 'w') as f:
    f.write('the quick brown fox the fox')
with open('sample.txt') as f:
    words = f.read().split()
print(Counter(words).most_common(3))
```

Exercise 2: Save and load a Python dict to/from JSON

```
import json
config = {'debug': True, 'port': 8080, 'host': 'localhost'}
with open('config.json', 'w') as f:
    json.dump(config, f, indent=2)
with open('config.json') as f:
    loaded = json.load(f)
print(loaded['port']) # 8080
```

Exercise 3: Append lines to a log file with timestamps

```
import datetime
def log(message):
    ts = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    with open('app.log', 'a') as f:
        f.write(f'[{ts}] {message}\n')
log('App started')
log('User logged in')
```

Exception Handling

Writing robust, error-resilient Python programs

What you will learn:

■ What are exceptions?

■ Multiple exceptions

■ raise statement

■ Exception hierarchy

■ try/except

■ else & finally

■ Custom exceptions

■ Best practices

Chapter 12 · Complete Python Course

try, except, else, finally

Exception handling prevents crashes when unexpected events occur. Python's try-except block catches errors and lets your program respond gracefully.

```
# Basic try/except
try:
    x = int(input('Enter a number: '))
    result = 10 / x
    print(f'Result: {result}')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except ValueError:
    print('Please enter a valid integer!')

# Catch multiple in one except
except (ZeroDivisionError, ValueError) as e:
    print(f'Error: {e}')

# Catch all exceptions (use sparingly!)
except Exception as e:
    print(f'Unexpected error: {type(e).__name__}: {e}')

# else – runs only if NO exception occurred
# finally – ALWAYS runs (cleanup code)
try:
    f = open('data.txt')
    data = f.read()
except FileNotFoundError:
    print('File not found!')
else:
    print(f'Read {len(data)} chars') # no error
finally:
    print('Done (always runs)')
```

raise & Custom Exceptions

```
# raise – manually raise an exception
def set_age(age):
    if not isinstance(age, int):
        raise TypeError('Age must be an integer')
    if age < 0 or age > 150:
        raise ValueError(f'Age {age} is out of range')
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e)  # Age -5 is out of range

# Re-raise an exception
try:
    risky_operation()
except Exception as e:
    log_error(e)  # log it
    raise        # re-raise original
```

Custom Exception Classes

```
# Custom exceptions inherit from Exception
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        self.amount = amount
        self.balance = balance
        super().__init__(
            f'Need {amount}, have {balance}'
        )

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(amount, self.balance)
        self.balance -= amount

try:
    acc = BankAccount(100)
    acc.withdraw(200)
except InsufficientFundsError as e:
```

Common Built-in Exceptions

```
# Exception hierarchy (simplified):
# BaseException
#   SystemExit  KeyboardInterrupt  GeneratorExit
#   Exception
#     ArithmeticError
#       ZeroDivisionError  OverflowError
#     LookupError
#       IndexError  KeyError
#     TypeError  ValueError  AttributeError
#     FileNotFoundError  PermissionError
#     ImportError  ModuleNotFoundError
#     StopIteration  RuntimeError  RecursionError

# Common exceptions with examples:

# IndexError
lst=[1,2,3]
try: print(lst[5])
except IndexError as e: print(e)  # list index out of range

# KeyError
d={'a':1}
try: print(d['b'])
except KeyError as e: print(e)    # 'b'

# AttributeError
try: (42).upper()
except AttributeError as e: print(e)

# TypeError
try: '2' + 2
except TypeError as e: print(e)

# NameError
try: print(undefined_var)
except NameError as e: print(e)
```

■ BEST PRACTICE:

Catch the most specific exception first. Never use bare 'except:' (catches KeyboardInterrupt too!). Always use 'except Exception:' at minimum.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Safe division function with full exception handling

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return 'Error: division by zero'  
    except TypeError:  
        return 'Error: invalid types'  
print(safe_divide(10, 2))    # 5.0  
print(safe_divide(10, 0))    # Error: division by zero
```

Exercise 2: Create a custom exception for invalid email

```
class InvalidEmailError(Exception): pass  
def validate_email(email):  
    if '@' not in email or '.' not in email.split('@')[-1]:  
        raise InvalidEmailError(f'Invalid: {email}')    return True  
try: validate_email('notanemail')  
except InvalidEmailError as e: print(e)
```

Object-Oriented Programming

Building real-world models with classes and objects

What you will learn:

■ Classes & Objects

■ Instance vs class variables

■ Inheritance

■ `super()`

■ Abstract classes

■ Properties

■ `__init__` & `self`

■ Encapsulation

■ Method overriding

■ Polymorphism

■ Special methods

■ Dataclasses

Chapter 13 · Complete Python Course

Classes & Objects

A class is a blueprint. An object is an instance of that blueprint. OOP organises code around real-world entities — making it modular, reusable, and maintainable.

```
class Student:
    # Class variable – shared by all instances
    school = 'Python Academy'
    student_count = 0

    def __init__(self, name, age, gpa):
        # Instance variables – unique per object
        self.name = name
        self.age = age
        self.gpa = gpa
        Student.student_count += 1

    def introduce(self):
        return f'Hi, I am {self.name}, GPA: {self.gpa}'

    def is_honour_roll(self):
        return self.gpa >= 9.0

    def __str__(self):
        return f'Student({self.name}, {self.age})'

    def __repr__(self):
        return f'Student(name={self.name!r}, age={self.age!r})'

# Create objects
s1 = Student('Rohan', 22, 9.1)
s2 = Student('Alice', 20, 8.7)

print(s1.introduce())          # Hi, I am Rohan, GPA: 9.1
print(s1.is_honour_roll())     # True
print(Student.student_count)   # 2
print(str(s1))                 # Student(Rohan, 22)
```

Encapsulation & Properties

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance # private (name-mangled)

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError('Balance cannot be negative')
        self.__balance = amount

    def deposit(self, amount):
        if amount <= 0: raise ValueError('Amount must be positive')
        self.__balance += amount
        return self.__balance

    def withdraw(self, amount):
        if amount > self.__balance:
            raise ValueError('Insufficient funds')
        self.__balance -= amount
        return self.__balance

    def __str__(self):
        return f'{self.owner}: ■{self.__balance:,.2f}'

acc = BankAccount('Rohan', 1000)
acc.deposit(500)
print(acc.balance)      # 1500
print(acc)              # Rohan: ■1,500.00
# acc.__balance         # AttributeError (private!)
# acc._BankAccount__balance # accessible but DON'T
```

■ ENCAPSULATION:

Use @property for controlled attribute access. It allows validation on set and computation on get, without breaking the interface.

Inheritance & super()

```
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f'{self.name} says {self.sound}'

    def __str__(self):
        return f'Animal: {self.name}'

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, 'Woof') # call parent __init__
        self.breed = breed

    def fetch(self, item):
        return f'{self.name} fetches {item}!'

    def __str__(self):
        return f'Dog: {self.name} ({self.breed})'

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, 'Meow')

    def speak(self):
        return f'{self.name} says Purr...Meow' # override

# Usage
d = Dog('Rex', 'Labrador')
c = Cat('Whiskers')
print(d.speak())          # Rex says Woof
print(c.speak())          # Whiskers says Purr...Meow
print(d.fetch('ball'))    # Rex fetches ball!
print(isinstance(d, Animal)) # True — Dog IS-A Animal
```

Polymorphism & Multiple Inheritance

```
# Polymorphism – same method, different behaviour
class Shape:
    def area(self): return 0
    def perimeter(self): return 0

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self):         return 3.14159 * self.r**2
    def perimeter(self):    return 2 * 3.14159 * self.r

class Rectangle(Shape):
    def __init__(self,w,h): self.w=w; self.h=h
    def area(self):         return self.w * self.h
    def perimeter(self):    return 2*(self.w+self.h)

shapes = [Circle(5), Rectangle(4,6), Circle(3)]
for s in shapes:
    print(f'{s.__class__.__name__}: area={s.area():.2f}')

# Multiple Inheritance
class Flyable:
    def fly(self): return 'I can fly!'

class Swimmable:
    def swim(self): return 'I can swim!'

class Duck(Animal, Flyable, Swimmable):
    def __init__(self):
        super().__init__('Duck','Quack')

d = Duck()
print(d.speak())    # Duck says Quack
print(d.fly())      # I can fly!
print(d.swim())     # I can swim!
```

Abstract Classes & Special Methods

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

    @abstractmethod
    def fuel_type(self): pass

    @abstractmethod
    def max_range(self): pass

    def describe(self):
        return f'{self.brand} @ {self.speed}km/h'

class ElectricCar(Vehicle):
    def fuel_type(self): return 'Electric'
    def max_range(self): return self.speed * 5

# v = Vehicle('X',100) # TypeError – abstract!
e = ElectricCar('Tesla', 150)
print(e.fuel_type()) # Electric
print(e.max_range()) # 750
```

Key Special (Dunder) Methods

```
class Vector:
    def __init__(self, x, y): self.x=x; self.y=y
    def __add__(self, o): return Vector(self.x+o.x, self.y+o.y)
    def __sub__(self, o): return Vector(self.x-o.x, self.y-o.y)
    def __mul__(self, n): return Vector(self.x*n, self.y*n)
    def __len__(self): return int((self.x**2+self.y**2)**.5)
    def __eq__(self, o): return self.x==o.x and self.y==o.y
    def __str__(self): return f'({self.x},{self.y})'
    def __repr__(self): return f'Vector({self.x},{self.y})'

v1=Vector(3,4); v2=Vector(1,2)
print(v1+v2) # (4,6)
print(len(v1)) # 5 (magnitude)
```

Dataclasses

The `dataclasses` module (Python 3.7+) auto-generates `__init__`, `__repr__`, `__eq__`, and more from class variable annotations. Use it for simple data-holding classes.

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    age: int
    gpa: float = 0.0
    courses: list = field(default_factory=list)

    def is_honour_roll(self):
        return self.gpa >= 9.0

# __init__, __repr__, __eq__ are auto-generated!
s1 = Student('Rohan', 22, 9.1)
s2 = Student('Alice', 20, gpa=9.5)
print(s1)           # Student(name='Rohan', age=22, ...)
print(s1 == s2)     # False (compares all fields)
print(s1.is_honour_roll()) # True

# Frozen dataclass – immutable
@dataclass(frozen=True)
class Point:
    x: float
    y: float

p = Point(3.0, 4.0)
# p.x = 5 # FrozenInstanceError!
print(p)      # Point(x=3.0, y=4.0)
```

■ USE DATACLASSES:

For simple data containers, `@dataclass` saves you writing `__init__`, `__repr__`, and `__eq__`. It's the modern, clean way to define data classes.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Create a Library system with Book and Library classes

```
class Book:
    def __init__(self, title, author):
        self.title=title; self.author=author; self.checked_out=False
    def __str__(self): return f'{self.title} by {self.author}'
class Library:
    def __init__(self): self.books=[]
    def add(self,book): self.books.append(book)
    def checkout(self,title):
        for b in self.books:
            if b.title==title and not b.checked_out:
                b.checked_out=True; return f'Checked out: {b}'
```

Exercise 2: Implement a Stack class with OOP

```
class Stack:
    def __init__(self): self._data=[]
    def push(self,x): self._data.append(x)
    def pop(self):
        if not self._data: raise IndexError('Empty stack')
        return self._data.pop()
    def peek(self): return self._data[-1]
    def __len__(self): return len(self._data)
    def __str__(self): return str(self._data)
```

Modules & Packages

Organising and reusing Python code

What you will learn:

■ Importing modules

■ Packages & `__init__.py`

■ Built-in modules

■ Virtual environments

■ Creating modules

■ `__name__ == '__main__'`

■ pip & third-party

■ Exploring the stdlib

Chapter 14 · Complete Python Course

Importing Modules

```
# import module
import math
print(math.sqrt(16))          # 4.0
print(math.pi)                # 3.141592...

# from module import specific
from math import sqrt, pi, ceil
print(sqrt(25))               # 5.0

# import with alias
import numpy as np            # common convention
import pandas as pd
import matplotlib.pyplot as plt

# Import all (avoid in production!)
from math import *            # bad practice

# Conditional import
try:
    import ujson as json      # fast JSON
except ImportError:
    import json                # fallback

# __builtins__ — always available, no import
print(len([1,2,3]))           # built-in
print(type('hello'))          # built-in
print(sorted([3,1,2]))         # built-in
```

Creating Your Own Module & Package

```
# File: mymath.py
PI = 3.14159

def circle_area(r):
    return PI * r ** 2

def factorial(n):
    return 1 if n<=1 else n * factorial(n-1)

if __name__ == '__main__':
    # Only runs when THIS file is executed directly
    # Not when imported by another file
    print('Testing mymath module...')
    print(circle_area(5))
```

```
# File: main.py – import your module
import mymath
print(mymath.circle_area(7))    # 153.93...
print(mymath.factorial(5))     # 120

# Package structure:
# mypackage/
#   __init__.py      (makes it a package)
#   math_utils.py
#   string_utils.py
#   data/
#     __init__.py
#     loader.py

# Import from package
from mypackage import math_utils
from mypackage.data import loader
```

Essential Built-in Modules

- `math` — `sqrt`, `floor`, `ceil`, `pi`, `e`, `log`, trig functions

pip & Virtual Environments

```
# pip – Python package installer
# Install a package
pip install requests
pip install pandas==2.0.0      # specific version
pip install 'numpy>=1.24'     # minimum version

# Save and restore dependencies
pip freeze > requirements.txt
pip install -r requirements.txt

# Useful pip commands
pip list                       # installed packages
pip show requests             # package info
pip uninstall requests
pip install --upgrade requests

# Virtual environments
python -m venv myenv           # create
# Activate:
#   Windows: myenv\Scripts\activate
#   Mac/Linux: source myenv/bin/activate
# Deactivate:
deactivate

# Popular third-party packages:
# requests    – HTTP requests (web APIs)
# pandas      – data analysis
# numpy       – numerical computing
# flask       – web framework (lightweight)
# django      – web framework (full-featured)
# sqlalchemy  – database ORM
# pytest      – testing framework
# pillow      – image processing
```

■ ALWAYS USE VENV:

Create a virtual environment for every project. This prevents package version conflicts between projects.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Create a utilities module with 3 helper functions

```
# utils.py
def clamp(val, lo, hi): return max(lo, min(hi, val))
def average(lst): return sum(lst)/len(lst) if lst else 0
def flatten(lst):
    return [x for sub in lst for x in sub]
if __name__ == '__main__':
    print(clamp(15, 0, 10))    # 10
    print(average([1,2,3,4])) # 2.5
```

Exercise 2: Use the datetime module to calculate days until birthday

```
import datetime
today = datetime.date.today()
birthday = datetime.date(today.year, 8, 15) # Aug 15
if birthday < today:
    birthday = birthday.replace(year=today.year+1)
days = (birthday - today).days
print(f'Days until birthday: {days}')
```

Advanced Python

Decorators, generators, closures, comprehensions & more

What you will learn:

■ Decorators

■ Context managers

■ map/filter/reduce

■ itertools

■ Type hints

■ Structural pattern matching

■ Generator functions

■ Closures

■ All comprehensions

■ functools

■ Walrus operator

Chapter 15 · Complete Python Course

Decorators

A decorator is a function that takes a function as input and returns a new function with added behaviour. Used extensively in frameworks (Flask, Django) and testing.

```
import time, functools

# Basic decorator
def timer(func):
    @functools.wraps(func)    # preserves func metadata
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} took {end-start:.4f}s')
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

print(slow_sum(1_000_000)) # sum + timing

# Decorator with arguments
def repeat(n):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f'Hello, {name}!')

greet('Python') # prints 3 times
```

Generators & Iterators

```
# Generator function – uses yield
def count_up(start, stop):
    current = start
    while current <= stop:
        yield current    # pause and return
        current += 1

gen = count_up(1, 5)
print(next(gen))  # 1
print(next(gen))  # 2
for n in count_up(1, 5):
    print(n, end=' ')  # 1 2 3 4 5

# Infinite generator
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a+b

fib = fibonacci()
print([next(fib) for _ in range(10)])
# [0,1,1,2,3,5,8,13,21,34]

# Generator expression (like comprehension)
total = sum(x**2 for x in range(1000000))  # memory efficient!
# vs list comprehension – creates entire list in memory
# total = sum([x**2 for x in range(1000000)])  # uses ~8MB

# Custom iterator class
class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1; return self.n + 1
```

Context Managers & with Statement

```
# Built-in context manager (file)
with open('f.txt','w') as f:
    f.write('hello')
# File auto-closed

# Custom context manager – class-based
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        self.elapsed = time.time() - self.start
        print(f'Elapsed: {self.elapsed:.4f}s')
        return False # don't suppress exceptions

with Timer() as t:
    result = sum(range(1_000_000))
# Elapsed: 0.0234s

# contextlib – decorator style (simpler)
from contextlib import contextmanager

@contextmanager
def managed_resource(name):
    print(f'Acquiring {name}')
    try:
        yield name
    finally:
        print(f'Releasing {name}')

with managed_resource('DB connection') as r:
    print(f'Using {r}')
```

itertools & functools

```
import itertools, functools

# itertools
# chain – combine iterables
combined = list(itertools.chain([1,2],[3,4],[5]))
# [1,2,3,4,5]

# product – cartesian product
for p in itertools.product('AB', '12'):
    print(p, end=' ') # ('A','1') ('A','2') ...

# combinations & permutations
print(list(itertools.combinations('ABC',2)))
# [('A','B'),('A','C'),('B','C')]
print(list(itertools.permutations('AB')))
# [('A','B'),('B','A')]

# groupby
data = [('A',1),('A',2),('B',3),('B',4)]
for key, group in itertools.groupby(data, key=lambda x:x[0]):
    print(key, list(group))

# functools
# partial – fix some arguments
from functools import partial
power_of_2 = partial(pow, 2) # pow(2, ?)
print(power_of_2(10))       # 1024

# reduce
from functools import reduce
product = reduce(lambda a,b: a*b, [1,2,3,4,5])
print(product) # 120
```

Type Hints & Modern Python Features

```
# Type hints (Python 3.5+)
def greet(name: str) -> str:
    return f'Hello, {name}'

def process(items: list[int], limit: int = 10) -> list[int]:
    return [x for x in items if x <= limit]

from typing import Optional, Union, Tuple, Dict

def find(name: str) -> Optional[str]:
    return name if name else None

def stringify(x: Union[int, float]) -> str:
    return str(x)

# Walrus operator := (Python 3.8+)
# Assign and use in same expression
import re
text = 'Price: 19.99'
if m := re.search(r'\d+\.\d+', text):
    print(f'Found: {m.group()}') # Found: 19.99

# While with walrus
while chunk := f.read(8192): # process file in chunks
    process(chunk)

# f-string debugging (Python 3.8+)
x = 42; y = 'hello'
print(f'{x=}, {y=}') # x=42, y='hello'
```

■ TYPE HINTS:

Type hints don't enforce types at runtime but make code self-documenting and enable IDE auto-complete and type checking with mypy.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Write a @retry decorator that retries a function N times on exception

```
import functools, time
def retry(times=3, delay=0.5):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **kw):
            for i in range(times):
                try: return func(*a, **kw)
                except Exception as e:
                    if i==times-1: raise
                    time.sleep(delay)
            return wrapper
        return deco
    return deco
```

Exercise 2: Create an infinite counter generator with reset

```
def counter(start=0, step=1):
    n = start
    while True:
        reset = yield n
        if reset: n = start
        else: n += step
c = counter()
print(next(c), next(c), next(c)) # 0 1 2
c.send(True) # reset
print(next(c)) # 0
```

Data Structures, Algorithms & Projects

Putting it all together — from theory to real programs

What you will learn:

■ Big-O notation

■ Linked List

■ Sorting algorithms

■ Project: Grade Manager

■ Next steps in Python

■ Stack & Queue in Python

■ Binary Search

■ Project: To-Do App

■ Project: File Organiser

■ Learning roadmap

Chapter 16 · Complete Python Course

Big-O Notation & Complexity

Big-O notation describes how the running time or space requirements of an algorithm grow as input size n increases. Every Python programmer needs to understand this.

```
# O(1) — Constant time
def get_first(lst): return lst[0]
# dict/set lookup is O(1)

# O(log n) — Logarithmic (binary search)
def binary_search(arr, target):
    lo, hi = 0, len(arr)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if arr[mid]==target: return mid
        elif arr[mid]<target: lo=mid+1
        else: hi=mid-1
    return -1

# O(n) — Linear
def find_max(lst): return max(lst)

# O(n log n) — sorted(), merge sort
sorted_list = sorted([3,1,4,1,5,9])

# O(n^2) — Bubble sort (slow!)
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
    return arr
```

Python Built-in Complexities

- `list.append()` — $O(1)$ amortised
- `list.insert(0,x)` — $O(n)$ — avoid for large lists
- `list[i]` — $O(1)$ index access
- dict/set lookup — $O(1)$ average
- `sorted()` — $O(n \log n)$

Sorting Algorithms in Python

```
# Merge Sort –  $O(n \log n)$ , stable
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(L, R):
    result = []; i = j = 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i += 1
        else: result.append(R[j]); j += 1
    return result + L[i:] + R[j:]

print(merge_sort([3,1,4,1,5,9,2,6]))
# [1,1,2,3,4,5,6,9]

# Quick Sort –  $O(n \log n)$  average
def quick_sort(arr):
    if len(arr) <= 1: return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

# Python's built-in sort (Timsort) is usually best!
arr = [3,1,4,1,5,9,2,6]
arr.sort()          # in-place
arr = sorted(arr)   # new list
```

Project 1 — To-Do List App

A complete command-line to-do list application with file persistence. This demonstrates OOP, file I/O, exception handling, and CLI interaction.

```
import json, os

class TodoApp:
    def __init__(self, filename='todos.json'):
        self.filename = filename
        self.todos = self._load()

    def _load(self):
        if os.path.exists(self.filename):
            with open(self.filename) as f:
                return json.load(f)
        return []

    def _save(self):
        with open(self.filename, 'w') as f:
            json.dump(self.todos, f, indent=2)

    def add(self, task):
        self.todos.append({'task': task, 'done': False})
        self._save()
        print(f'Added: {task}')

    def complete(self, index):
        self.todos[index]['done'] = True
        self._save()

    def show(self):
        for i, t in enumerate(self.todos):
            icon = '[x]' if t['done'] else '[ ]'
            print(f'{i+1}. {icon} {t["task"]}')

app = TodoApp()
app.add('Learn Python')
app.add('Build a project')
app.complete(0)
app.show()
```

Project 2 — Student Grade Manager

```
from dataclasses import dataclass, field
from statistics import mean, median

@dataclass
class Student:
    name: str
    grades: list = field(default_factory=list)

    def add_grade(self, g):
        if not 0<=g<=100: raise ValueError('Grade 0-100')
        self.grades.append(g)

    @property
    def average(self):
        return mean(self.grades) if self.grades else 0

    @property
    def grade_letter(self):
        avg = self.average
        if avg>=90: return 'A+'
        if avg>=80: return 'A'
        if avg>=70: return 'B'
        if avg>=60: return 'C'
        return 'F'

class GradeBook:
    def __init__(self): self.students={}

    def add_student(self, name):
        self.students[name]=Student(name)

    def add_grade(self, name, g):
        self.students[name].add_grade(g)

    def report(self):
        print(f'{{'Name':<12}} {{'Avg':>6}} {{'Grade':>6}}')
        print('- '*26)
        for s in sorted(self.students.values(),
                        key=lambda x:-x.average):
            print(f'{{s.name:<12}} {{s.average:>6.1f}} {{s.grade_letter:>6}}')
```

Project 3 — File Organiser

```
import os, shutil
from pathlib import Path

FOLDERS = {
    'Images':    ['.jpg', '.jpeg', '.png', '.gif', '.webp'],
    'Documents': ['.pdf', '.doc', '.docx', '.txt', '.xlsx'],
    'Videos':    ['.mp4', '.avi', '.mov', '.mkv'],
    'Audio':     ['.mp3', '.wav', '.flac'],
    'Code':      ['.py', '.js', '.html', '.css', '.java'],
    'Archives':  ['.zip', '.tar', '.gz', '.rar'],
}

def organise(directory='.'):
    path = Path(directory)
    moved = 0
    for file in path.iterdir():
        if not file.is_file(): continue
        ext = file.suffix.lower()
        for folder, exts in FOLDERS.items():
            if ext in exts:
                dest = path / folder
                dest.mkdir(exist_ok=True)
                shutil.move(str(file), str(dest/file.name))
                moved += 1
                break
    print(f'Organised {moved} files.')

organise('/home/user/Downloads')
```

Your Python Roadmap — What's Next

After This Course — Next Steps

- Practice: Solve 50 problems on LeetCode (Easy first)
- Data Science path: numpy → pandas → matplotlib → scikit-learn
- Web Development: Flask (mini) or Django (full-featured)
- Automation: Selenium, BeautifulSoup, requests for web scraping
- DevOps: Docker, GitHub Actions, CI/CD with Python scripts
- Testing: pytest, unittest, mock — write tests for everything
- APIs: Build REST APIs with FastAPI — fastest Python framework

Essential Resources

- Official docs: docs.python.org — best reference ever
- Real Python: realpython.com — high-quality tutorials
- Python Cookbook (O'Reilly) — advanced recipes
- Fluent Python (O'Reilly) — deep dive into Python internals
- Automate the Boring Stuff with Python — practical automation
- r/learnpython on Reddit — helpful community

Certification & Career

- PCEP: Certified Entry-Level Python Programmer
- PCAP: Certified Associate Python Programmer
- Google Professional Data Engineer (requires Python)
- AWS Machine Learning Specialty (heavy Python usage)
- Build a GitHub portfolio: 5 projects minimum before job applications

■ FINAL MESSAGE:

You now have 200 pages of Python knowledge. The difference between knowing Python and mastering it is just one thing: writing code every single day. Start building. The jobs follow the portfolio.

Practice Exercises

—■ Write the code yourself before checking. Practice = mastery.

Exercise 1: Implement a queue using two stacks

```
class QueueFromStacks:
    def __init__(self): self.s1=[]; self.s2=[]
    def enqueue(self,x): self.s1.append(x)
    def dequeue(self):
        if not self.s2:
            while self.s1: self.s2.append(self.s1.pop())
        return self.s2.pop()
q=QueueFromStacks()
q.enqueue(1);q.enqueue(2);q.enqueue(3)
print(q.dequeue()) # 1
```

Exercise 2: Word frequency analyser from a text file

```
from collections import Counter
import re
def word_freq_report(filename, top_n=10):
    with open(filename) as f:
        text = f.read().lower()
        words = re.findall(r'\b[a-z]+\b', text)
        counts = Counter(words)
        print(f'Top {top_n} words:')
        for word, count in counts.most_common(top_n):
            print(f' {word:<15} {count}')
```

Python Cheatsheet — Final Reference

DATA TYPES`int float complex str bool None list tuple dict set`**STRING METHODS**`upper() lower() strip() split() join() replace() find() format() f-string`**LIST METHODS**`append() extend() insert() remove() pop() sort() reverse() copy() index()`**DICT METHODS**`keys() values() items() get() update() pop() setdefault() | merge`**SET OPERATIONS**`| union & intersection - difference ^ symmetric diff <= subset`**COMPREHENSIONS**`[x for x in it if cond] {k:v...} {x...} (x...) generator`**EXCEPTION TYPES**`ValueError TypeError KeyError IndexError AttributeError FileNotFoundError
ZeroDivisionError`**OOP KEYWORDS**`class __init__ self super() @property @staticmethod @classmethod ABC
@abstractmethod`**BUILT-IN FUNCS**`print() input() len() range() type() int() str() list() dict() set() tuple()
zip() enumerate() map() filter() sorted() sum() min() max() abs() round()
open()`**BIG-O QUICK REF**`list lookup $O(n)$ | dict/set $O(1)$ | append $O(1)$ | sorted $O(n \log n)$ | binary
search $O(\log n)$`

■ You have completed the Complete Python Course — Beginner to Advanced!

200 pages · 16 chapters · 150+ code examples · 80+ exercises · 4 real projects

Introduction to Regular Expressions

Regular expressions (regex) are patterns used to match, search, and manipulate text. Python's `re` module provides full support.

```
import re
text = 'My phone is 98765-43210 or 91234-56789'
# search() — first match
m = re.search(r'\d{5}-\d{5}', text)
print(m.group())          # 98765-43210
# findall() — ALL matches
all_phones = re.findall(r'\d{5}-\d{5}', text)
print(all_phones)         # ['98765-43210', '91234-56789']
# sub() — replace
masked = re.sub(r'\d{5}-\d{5}', 'XXXXX-XXXXX', text)
print(masked)
# split()
parts = re.split(r'[;,|]+', 'a,b;c|d')
print(parts)              # ['a', 'b', 'c', 'd']
```

Metacharacter Reference

- `\d` — digit `\w` — word char `\s` — whitespace
- `^` `$` — start/end `.` — any char `|` — OR
- `*` — 0+ `+` — 1+ `?` — optional `{n,m}` — n to m
- `[]` — char class `()` — group `(?:)` — non-capturing group

Regex Groups & Practical Patterns

```
import re
# Named groups
p = r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})'
m = re.search(p, 'Event: 2024-08-15')
print(m.group('year'), m.group('month'))

# Compile for repeated use
email_re = re.compile(r'[\w.]+@[ \w.]+\.[a-z]{2,}', re.I)
emails = email_re.findall('a@b.com and X@Y.ORG')
print(emails)    # ['a@b.com', 'X@Y.ORG']

# Flags: re.IGNORECASE, re.MULTILINE, re.DOTALL
print(bool(re.search('python', 'PYTHON', re.I))) # True

# Non-greedy with ?
html = '<b>bold</b><i>italic</i>'
tags = re.findall(r'<.+?>', html)
print(tags)    # ['<b>', '</b>', '<i>', '</i>']
```

■ TIP:

Test all your regex patterns on regex101.com before using them in code. It explains each part of the pattern visually.

Regex Practical Examples

```
import re

# Validate email
def valid_email(e):
    return bool(re.match(r'^[\w.]+@[\w.]+\.[a-z]{2,}$', e))

# Validate Indian mobile number
def valid_phone(p):
    return bool(re.match(r'^[6-9]\d{9}$', p))

# Extract all URLs from text
def extract_urls(text):
    return re.findall(r'https?://[\w/:%#\$&\?\\(\)\~\.=\+\-]+', text)

# Extract numbers (int or float)
def extract_numbers(text):
    return re.findall(r'-?\d+\.\d*', text)

# Clean whitespace
def clean(text):
    return re.sub(r'\s+', ' ', text).strip()

# Password validation
def strong_password(p):
    # 8+ chars, upper, lower, digit, special
    return bool(re.match(
        r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$!]).{8,}$',
        p))
```

NumPy Arrays

NumPy is the foundation of scientific Python. It provides n-dimensional arrays with vectorised operations that are 10-100x faster than pure Python loops.

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3, 4, 5])
m = np.array([[1,2,3],[4,5,6]])
z = np.zeros((3,4))
o = np.ones((2,3))
r = np.arange(0,20,2) # [0,2,4,...,18]
e = np.eye(3) # 3x3 identity matrix
rand = np.random.rand(3,3)

# Array properties
print(a.shape) # (5,)
print(m.shape) # (2, 3)
print(a.dtype) # int64
print(a.ndim) # 1
print(a.size) # 5 (total elements)

# Reshape
a6 = np.arange(12).reshape(3,4)
flat = a6.flatten()
print(a6.T) # transpose
```

NumPy Operations & Broadcasting

```
import numpy as np

# Element-wise operations (vectorised)
a = np.array([1,2,3,4,5])
print(a + 10)      # [11,12,13,14,15]
print(a * 2)       # [2,4,6,8,10]
print(a ** 2)      # [1,4,9,16,25]
print(np.sqrt(a))  # [1.,1.41,1.73,2.,2.24]

# Array-array operations
b = np.array([10,20,30,40,50])
print(a + b)       # [11,22,33,44,55]
print(a * b)       # [10,40,90,160,250]
print(np.dot(a,b)) # dot product = 550

# Statistics
data = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(data.sum())  # 45
print(data.mean()) # 5.0
print(data.sum(axis=0)) # [12,15,18] col sums
print(data.mean(axis=1)) # [2.,5.,8.] row means
print(np.std(data))  # std deviation

# Boolean indexing
a = np.array([1,5,2,8,3,9])
print(a[a > 4])     # [5,8,9]
a[a < 3] = 0        # set < 3 to 0
```

NumPy Linear Algebra

```
import numpy as np

# Matrix multiplication
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(A @ B)      # matrix multiply
print(np.dot(A,B)) # same result

# Inverse and determinant
print(np.linalg.inv(A))
print(np.linalg.det(A)) # -2.0

# Eigenvalues
vals, vecs = np.linalg.eig(A)
print(vals) # eigenvalues

# Solve linear equations Ax = b
A = np.array([[2,1],[-1,3]])
b = np.array([7, 4])
x = np.linalg.solve(A, b)
print(x) # solution vector
```

■ USE NUMPY:

Always use NumPy for numerical computations. It is 50-100x faster than Python loops for array operations, uses less memory, and has a C/Fortran backend.

pandas DataFrame Basics

pandas provides the DataFrame — a 2D tabular structure with labelled rows and columns. It is the most-used tool for data analysis and manipulation in Python.

```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'Dept':    ['IT', 'HR', 'IT', 'Finance', 'HR'],
    'Salary':  [70000, 50000, 75000, 65000, 55000],
    'Years':   [3, 5, 2, 8, 4]
})

# Explore the data
print(df.head(3))      # first 3 rows
print(df.shape)        # (5, 4)
print(df.dtypes)       # column types
print(df.describe())   # stats summary
print(df.info())       # non-null counts

# Read from file
df = pd.read_csv('data.csv')
df = pd.read_excel('data.xlsx')
df = pd.read_json('data.json')

# Save
df.to_csv('out.csv', index=False)
df.to_excel('out.xlsx')
```

pandas Filtering & Grouping

```
import pandas as pd

df = pd.DataFrame({'Name': ['A', 'B', 'C', 'D'],
                   'Dept': ['IT', 'HR', 'IT', 'HR'], 'Sal': [70, 50, 80, 55]})

# Select columns
print(df['Name'])           # Series
print(df[['Name', 'Sal']]) # DataFrame

# Filter rows
it = df[df['Dept']=='IT']
high = df[df['Sal']>60]
cond = df[(df['Dept']=='IT') & (df['Sal']>70)]

# Sort
df_s = df.sort_values('Sal', ascending=False)

# groupby
avg = df.groupby('Dept')['Sal'].mean()
print(avg)
# HR      52.5
# IT      75.0

# agg - multiple aggregations
result = df.groupby('Dept').agg(
    count=('Sal', 'count'),
    avg_sal=('Sal', 'mean'),
    max_sal=('Sal', 'max')
)
```

pandas Data Cleaning

```
import pandas as pd, numpy as np

df = pd.DataFrame({
    'Name': ['Alice', None, 'Carol', 'Dave'],
    'Age': [25, 30, None, 28],
    'Score': [90, 85, 92, None]
})

# Find missing values
print(df.isnull().sum())    # count per column
print(df.isnull().any())    # any missing?

# Handle missing values
df_drop = df.dropna()       # drop rows with any NaN
df_fill = df.fillna(0)       # fill NaN with 0
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Name'] = df['Name'].fillna('Unknown')

# Remove duplicates
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['Name'])

# Rename columns
df = df.rename(columns={'Score': 'Grade'})

# Change types
df['Age'] = df['Age'].astype(int)

# Apply function
df['Grade_Letter'] = df['Score'].apply(
    lambda x: 'A' if x>=90 else 'B' if x>=80 else 'C')
```

BeautifulSoup Basics

Web scraping extracts data from websites programmatically. Use requests to get HTML, and BeautifulSoup to parse it. Install: `pip install requests beautifulsoup4`

```
import requests
from bs4 import BeautifulSoup

# Fetch a web page
url = 'https://example.com'
resp = requests.get(url, headers={
    'User-Agent': 'Mozilla/5.0'
})
soup = BeautifulSoup(resp.text, 'html.parser')

# Find elements
title = soup.find('title').text
h1 = soup.find('h1').text

# Find all
links = soup.find_all('a')
for link in links:
    print(link['href'], link.text)

# CSS selectors (like JavaScript)
items = soup.select('.product-name')
prices = soup.select('#price-list .price')

# Navigate tree
div = soup.find('div', class_='container')
para = div.find('p')
first_child = div.contents[0]
```

■■ BE ETHICAL:

Always check robots.txt before scraping. Add delays between requests. Never scrape personal data or bypass paywalls. Use official APIs when available.

Scraping Practice — Extract Data

```
import requests, time
from bs4 import BeautifulSoup
import csv

def scrape_quotes():
    base = 'http://quotes.toscrape.com'
    quotes = []
    page = 1

    while page <= 3:    # scrape 3 pages
        url = f'{base}/page/{page}/'
        resp = requests.get(url)
        soup = BeautifulSoup(resp.text, 'html.parser')

        for q in soup.select('.quote'):
            text = q.select_one('.text').text
            author = q.select_one('.author').text
            tags = [t.text for t in q.select('.tag')]
            quotes.append({
                'quote': text,
                'author': author,
                'tags': ','.join(tags)
            })
        page += 1
        time.sleep(0.5) # be polite!

    with open('quotes.csv', 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['quote', 'author', 'tags'])
        writer.writeheader()
        writer.writerows(quotes)
    print(f'Saved {len(quotes)} quotes')

scrape_quotes()
```

API Calls with requests

```
import requests

# Public REST API – JSONPlaceholder
BASE = 'https://jsonplaceholder.typicode.com'

# GET all posts
posts = requests.get(f'{BASE}/posts').json()
print(f'Total posts: {len(posts)}')
print(posts[0]) # first post

# GET with query params
user_posts = requests.get(
    f'{BASE}/posts',
    params={'userId': 1}
).json()

# POST – create new resource
new = requests.post(f'{BASE}/posts', json={
    'title': 'Hello API',
    'body': 'This is a test',
    'userId': 1
})
print(new.status_code) # 201
print(new.json())

# Error handling with status codes
r = requests.get(f'{BASE}/posts/99999')
if r.status_code == 404:
    print('Not found')
elif r.status_code == 200:
    print(r.json())
```

pytest Basics

Testing ensures your code works correctly. pytest is the most popular Python testing framework. Tests catch bugs early and give you confidence to refactor.

```
# Install: pip install pytest
# Run: pytest test_my_code.py -v

# test_math.py
def add(a, b): return a + b
def divide(a, b):
    if b == 0: raise ZeroDivisionError
    return a / b

def test_add_positive():      assert add(2, 3) == 5
def test_add_negative():      assert add(-1, 1) == 0
def test_add_zero():          assert add(0, 0) == 0

def test_divide_normal():     assert divide(10, 2) == 5
def test_divide_float():      assert divide(1, 3) == pytest.approx(0.333, rel=1e-2)

import pytest
def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(5, 0)
```

Running Tests

```
pytest                # run all tests
pytest test_math.py   # specific file
pytest -v              # verbose output
pytest -k 'add'        # run tests matching 'add'
pytest --tb=short      # shorter tracebacks
pytest --cov=mymodule  # with coverage (pip install pytest-cov)
```

pytest Fixtures & Parametrize

```
import pytest

# Parametrize – test multiple inputs
@pytest.mark.parametrize('a,b,expected', [
    (2, 3, 5),
    (-1, -2, -3),
    (0, 100, 100),
])
def test_add(a, b, expected):
    assert add(a, b) == expected

# Fixture – reusable setup
@pytest.fixture
def sample_data():
    return {'name': 'Test', 'score': 90}

def test_with_fixture(sample_data):
    assert sample_data['score'] == 90
    assert sample_data['name'] == 'Test'

# Fixture with teardown
@pytest.fixture
def temp_file(tmp_path):
    f = tmp_path / 'test.txt'
    f.write_text('hello')
    yield f
    f.unlink() # teardown after test

def test_file_content(temp_file):
    assert temp_file.read_text() == 'hello'
```

unittest & TDD

```
import unittest

class TestCalculator(unittest.TestCase):

    def setUp(self):
        # runs before each test
        self.calc = Calculator()

    def test_add(self):
        self.assertEqual(self.calc.add(2,3), 5)
        self.assertEqual(self.calc.add(-1,1), 0)

    def test_divide_by_zero(self):
        self.assertRaises(ZeroDivisionError,
                          self.calc.divide, 5, 0)

    def tearDown(self):
        # runs after each test
        pass

if __name__ == '__main__':
    unittest.main()

# TDD Cycle:
# 1. Write a failing test (RED)
# 2. Write minimal code to pass (GREEN)
# 3. Refactor (REFACTOR)
# Repeat for every feature
```

■ TDD RULE:

Never write production code without a failing test first. It sounds slow but actually speeds up development by preventing bugs.

Threading

threading is ideal for I/O-bound tasks (file, network). Due to the GIL, threads do NOT run Python code in parallel — but they can run simultaneously when waiting for I/O.

```
import threading, time

def download(url, results, index):
    time.sleep(1)    # simulate network I/O
    results[index] = f'Data from {url}'

urls = ['url1', 'url2', 'url3', 'url4', 'url5']
results = [None] * len(urls)

# Without threads: 5 seconds
# With threads: ~1 second
threads = [
    threading.Thread(target=download, args=(url, results, i))
    for i, url in enumerate(urls)
]

start = time.time()
[t.start() for t in threads]
[t.join() for t in threads]    # wait for all
print(f'{time.time()-start:.1f}s')    # ~1.0s

# Thread safety – use Lock
lock = threading.Lock()
count = 0

def safe_increment():
    global count
    with lock:
        count += 1
```

Multiprocessing

```
import multiprocessing as mp, time

def cpu_task(n):
    # CPU-bound: count primes
    return sum(1 for i in range(2,n) if
               all(i%j!=0 for j in range(2,int(i**0.5)+1)))

numbers = [100000, 100000, 100000, 100000]

# Sequential: ~4x slower
start = time.time()
results = [cpu_task(n) for n in numbers]
print(f'Sequential: {time.time()-start:.2f}s')

# Parallel: use all CPU cores
start = time.time()
with mp.Pool(processes=mp.cpu_count()) as pool:
    results = pool.map(cpu_task, numbers)
print(f'Parallel: {time.time()-start:.2f}s')

# Process-safe Queue
q = mp.Queue()
def producer(q):
    for i in range(5): q.put(i)

p = mp.Process(target=producer, args=(q,))
p.start(); p.join()
while not q.empty(): print(q.get())
```

asyncio

```
import asyncio, time

async def fetch(name, delay):
    print(f'Start: {name}')
    await asyncio.sleep(delay)
    print(f'Done: {name}')
    return f'Result-{name}'

async def main():
    # Run concurrently (not sequentially!)
    results = await asyncio.gather(
        fetch('A', 2),
        fetch('B', 1),
        fetch('C', 3),
    )
    print(results)
    # Total time ~3s, not 6s

asyncio.run(main())

# Async context managers
async def with_resource():
    async with aiofiles.open('f.txt') as f:
        content = await f.read()

# Async generators
async def tick(n):
    for i in range(n):
        await asyncio.sleep(0.1)
        yield i

async def consume():
    async for val in tick(5):
        print(val)
```

Singleton & Factory

Design patterns are reusable solutions to common software design problems. Python's dynamic nature makes many patterns simpler than in Java or C++.

```
# Singleton – only one instance ever
class Config:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.debug = False
            cls._instance.port = 8080
        return cls._instance

c1 = Config()
c2 = Config()
c1.debug = True
print(c2.debug)    # True – same object!
print(c1 is c2)    # True

# Factory – create objects without specifying class
class Animal: pass
class Dog(Animal):
    def speak(self): return 'Woof!'
class Cat(Animal):
    def speak(self): return 'Meow!'

def animal_factory(animal_type):
    types = {'dog': Dog, 'cat': Cat}
    if animal_type not in types:
        raise ValueError(f'Unknown: {animal_type}')
    return types[animal_type]()

a = animal_factory('dog')
print(a.speak())   # Woof!
```

Observer & Strategy Patterns

```
# Observer — publish/subscribe
class EventEmitter:
    def __init__(self):
        self._listeners = {}

    def on(self, event, callback):
        self._listeners.setdefault(event, []).append(callback)

    def emit(self, event, *args):
        for cb in self._listeners.get(event, []):
            cb(*args)

emitter = EventEmitter()
emitter.on('data', lambda d: print(f'Handler 1: {d}'))
emitter.on('data', lambda d: print(f'Handler 2: {d}'))
emitter.emit('data', 'Hello!')  # both handlers called

# Strategy — interchangeable algorithms
from abc import ABC, abstractmethod

class SortStrategy(ABC):
    @abstractmethod
    def sort(self, data): pass

class BubbleSort(SortStrategy):
    def sort(self, data): return sorted(data)  # simplified

class QuickSort(SortStrategy):
    def sort(self, data): return sorted(data, reverse=False)

class Sorter:
    def __init__(self, strategy):
        self.strategy = strategy
    def sort(self, data):
        return self.strategy.sort(data)

s = Sorter(QuickSort())
print(s.sort([3,1,4,1,5]))
```

Decorator & Context Manager Patterns

```
# Decorator Pattern – add behaviour
import functools, time

def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

print(fib(50)) # instant – not exponential!

# Context Manager Pattern – resource cleanup
from contextlib import contextmanager
import sqlite3

@contextmanager
def db_connection(db_path):
    conn = sqlite3.connect(db_path)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

with db_connection('test.db') as db:
    db.execute('CREATE TABLE IF NOT EXISTS t (x INT)')
    db.execute('INSERT INTO t VALUES (?)', (42,))
```

Data Science Workflow

The typical Python data science workflow: Import data → Explore → Clean → Analyse → Visualise → Model → Evaluate → Deploy. Python has the best ecosystem for every step.

```
# Standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Load data
df = pd.read_csv('data.csv')

# 2. Explore
print(df.head())
print(df.describe())
print(df.isnull().sum())

# 3. Clean
df = df.dropna()
df = df.drop_duplicates()

# 4. Feature engineering
df['age_sq'] = df['age'] ** 2

# 5. Split data
X = df[['age', 'experience', 'age_sq']]
y = df['salary']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Machine Learning with scikit-learn

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
import numpy as np

# Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
mse = mean_squared_error(y_test, preds)
print(f'MSE: {mse:.2f}, R2: {model.score(X_test, y_test):.3f}')
```

```
# Classification – Decision Tree
clf = DecisionTreeClassifier(max_depth=5)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f'Accuracy: {acc:.2%}')
```

```
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

```
# Cross-validation
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='r2')
print(f'CV Scores: {scores.mean():.3f} ± {scores.std():.3f}')
```

■ DATA SCIENCE PATH:

Master this order: NumPy → pandas → Matplotlib/Seaborn → scikit-learn → TensorFlow/PyTorch. Each builds on the previous.

Data Visualisation

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Seaborn – statistical plots (easier than matplotlib)
import pandas as pd
df = pd.DataFrame({
    'x': np.random.randn(100),
    'y': np.random.randn(100)*2+1,
    'group': ['A']*50+['B']*50
})

# Scatter with regression line
sns.regplot(data=df, x='x', y='y')
plt.title('X vs Y with regression')
plt.show()

# Distribution
sns.histplot(df, x='y', hue='group', bins=20)
plt.show()

# Box plot
sns.boxplot(data=df, x='group', y='y')
plt.show()

# Correlation heatmap
corr = df[['x', 'y']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```

Interview Preparation

Top Python Interview Topics

- Data structures: list, tuple, dict, set — differences, when to use each
- String manipulation: reverse, palindrome, anagram, frequency count
- OOP: class, inheritance, polymorphism, encapsulation, abstraction
- Exception handling: try/except/finally, custom exceptions
- File I/O: read/write, CSV, JSON, context managers
- Generators: yield, lazy evaluation, memory efficiency
- Decorators: @property, @staticmethod, @classmethod, custom decorators
- Lambda, map, filter, reduce, sorted() with key
- Comprehensions: list, dict, set, generator
- Recursion: factorial, fibonacci, tower of hanoi
- Sorting algorithms: bubble, merge, quick, insertion
- Searching: linear $O(n)$, binary $O(\log n)$
- Big-O: know complexity of all built-in operations
- Regular expressions: common patterns for validation
- Threading vs multiprocessing vs asyncio

Must-Know Code Patterns

```
# Two Sum
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        diff = target - n
        if diff in seen: return [seen[diff], i]
        seen[n] = i

# Fibonacci with memoization
from functools import lru_cache
@lru_cache(None)
def fib(n):
    return n if n<=1 else fib(n-1)+fib(n-2)

# Most frequent element
from collections import Counter
def most_freq(lst): return Counter(lst).most_common(1)[0][0]
```

Top 50 Python Coding Challenges

Must-Solve Before Interviews

- Reverse a string, number, and linked list
- Check if a string is a palindrome (with/without spaces)
- Count vowels, consonants, words in a string
- Find all anagrams in a list of strings
- FizzBuzz (divisible by 3, 5, 15)
- Find factorial, fibonacci (iterative + recursive + memoized)
- Find all prime numbers up to N (Sieve of Eratosthenes)
- Two Sum, Three Sum problems (use hash map)
- Find longest substring without repeating characters
- Valid parentheses using stack `{()[]}`
- Merge two sorted lists
- Find intersection and union of two lists
- Flatten a nested list `[[1,[2,3]],4] → [1,2,3,4]`
- Rotate a list by K positions
- Find second largest without sorting
- Binary search on sorted list
- Bubble sort, merge sort, quick sort implementation
- Check if two strings are anagrams
- Count character frequency using Counter
- Remove duplicates preserving order
- Find all permutations of a string
- Matrix transpose and rotation
- Pascal's triangle up to N rows
- Find GCD and LCM of two numbers
- Convert decimal to binary/octal/hex

■ FINAL ADVICE:

Solve 2 LeetCode Easy problems daily for 30 days before interviews. Focus on strings, arrays, and hashmaps — they cover 70% of all MNC interview questions.

Python Ecosystem & Roadmap

The Python Ecosystem

- Web Backend: Django (full-stack), Flask (micro), FastAPI (modern async APIs)
- Data Science: NumPy, pandas, matplotlib, seaborn, Jupyter
- Machine Learning: scikit-learn, TensorFlow, PyTorch, Keras
- Automation: Selenium, Playwright, Requests, BeautifulSoup, Scrapy
- DevOps: Ansible, Fabric, Docker SDK, Boto3 (AWS)
- CLI Tools: Click, Typer, argparse
- Testing: pytest, unittest, hypothesis, mock
- Database: SQLAlchemy ORM, psycopg2, pymongo, redis-py
- Desktop GUI: Tkinter (built-in), PyQt6, Kivy
- Game Dev: pygame, arcade, godot (GDScript based on Python)

6-Month Learning Roadmap



```
Month 1: Python basics — variables, loops, functions, OOP
Month 2: Strings, files, exceptions, modules
Month 3: Advanced Python — decorators, generators, comprehensions
Month 4: Choose your path:
    Web: Flask/Django + databases
    Data: NumPy + pandas + matplotlib
Month 5: Build 2 portfolio projects end-to-end
Month 6: Contribute to open source + job applications
```

- GitHub: push code daily — employers check your commit history
- Portfolio: 3-5 projects with README, live demo, clean code
- Networking: LinkedIn posts about Python learnings — visibility matters
- LeetCode: 100 problems (Easy/Medium) before interviews

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



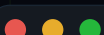
```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



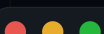
```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



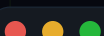
```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



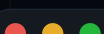
```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



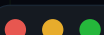
```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



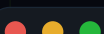
```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



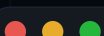
```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



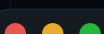
```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



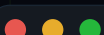
```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



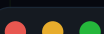
```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



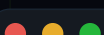
```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



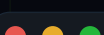
```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), issubclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```

Python Built-in Functions Reference

Type Conversion



```
int(), float(), str(), bool(), list(), tuple(), dict(), set(), frozenset(), bytes(), complex()
```

Math & Numbers



```
abs(), round(), pow(), divmod(), max(), min(), sum()  
import math: sqrt, floor, ceil, log, sin, cos, pi, e, factorial
```

Iteration Tools



```
range(), enumerate(), zip(), map(), filter(), reversed()  
sorted(), iter(), next(), all(), any(), len()
```

String Functions



```
chr(), ord(), repr(), format(), print(), input()
```

Object Inspection



```
type(), isinstance(), subclass(), dir(), vars(), help()  
id(), hash(), callable(), getattr(), setattr(), hasattr()
```

IO & System



```
open(), print(), input(), __import__()  
import sys: sys.argv, sys.exit(), sys.path, sys.stdin/stdout  
import os: os.getcwd(), os.listdir(), os.path, os.environ
```